

Priručnik: Documents-as-Code

Test / FGAG

Table of contents

Priručnik: Docs-as-Code - radni tok	8
Predgovor	8
Poglavlje 1 - Uvod: Nova paradigma rada s dokumentacijom	8
Preduvjeti za cijeli priručnik.....	8
Alati koje ćemo instalirati	8
1.1 Stari način rada – i zašto više ne funkcionira	9
1.2 Novi način rada – Docs-as-Code.....	9
1.3 Usporedba starog i novog načina	10
1.4 Tijek rada – kako to izgleda u praksi	10
1.5 Uloga svakog alata	11
Git	11
GitHub.....	11
Obsidian	11
Quarto	12
1.6 Zašto je ovo bolje? – Konkretno prednosti.....	12
1.7 Što ćeš naučiti u ovom priručniku	12
✓ Što smo naučili u ovom poglavlju.....	13
Poglavlje 2 - Instalacija i postavljanje okruženja.....	13
2.1 Git – instalacija.....	13
Što je Git i zašto ga instaliramo?	13
Korak 1: Preuzimanje	14
Korak 2: Instalacija (Windows).....	14
Korak 3: Provjera instalacije	14
Korak 4: Konfiguracija identiteta	14
2.2 Obsidian – instalacija.....	15
Što je Obsidian?	15
Korak 1: Preuzimanje	15
Korak 2: Instalacija	15

Korak 3: Početni ekran	15
2.3 Quarto – instalacija.....	16
Što je Quarto?	16
Korak 1: Preuzimanje	16
Korak 2: Instalacija	16
Korak 3: Provjera instalacije	16
2.4 GitHub – kreiranje računa.....	16
Što je GitHub?	16
Korak 1: Registracija	16
Korak 2: Traženje pristupa repozitoriju	17
2.5 Personal Access Token (PAT) – postavljanje autentikacije.....	17
Zašto nam treba PAT?	17
Korak 1: Kreiranje PAT-a na GitHubu	17
Korak 2: Konfiguracija tokena	17
Korak 3: Kopiranje i pohrana tokena	18
Korak 4: Pohrana tokena	18
2.6 .gitignore – što dijelimo, a što ne	18
Što je .gitignore?.....	18
Zašto je to važno za Obsidian?.....	18
Primjer .gitignore za ovaj projekt	19
✅ Što smo naučili u ovom poglavlju.....	19
Poglavlje 3 - Obsidian Git plugin: instalacija i konfiguracija	20
3.1 Pokretanje Obsidiana i privremeni vault	20
3.2 Omogućavanje Community plugina	20
3.3 Instalacija Obsidian Git plugina	21
3.4 Konfiguracija plugina – autentikacija	21
3.5 Konfiguracija plugina – automatizacija	22
Automatski pull	22
Commit poruke	22
3.6 Pregled sučelja plugina	23
Source Control panel (bočna traka)	23
Gumbi koje ćeš koristiti svaki dan	23
Status bar (statusna traka)	23

3.7 Što plugin radi automatski, a što ručno	23
✓ Što smo naučili u ovom poglavlju.....	24
Poglavlje 4 - Kloniranje repozitorija i kreiranje Obsidian vaulta	24
4.1 Što je repozitorij i što znači “kloniranje”?.....	25
4.2 Kloniranje repozitorija na računalo	25
Korak po korak:	25
4.3 Otvaranje kloniranog foldera kao Obsidian “Vault”	26
Korak po korak:	26
4.4 Razumijevanje strukture repozitorija	26
✓ Što smo naučili u ovom poglavlju.....	27
Poglavlje 5 - Podjela rada i timska suradnja	27
5.1 Noćna mora zvana “Merge Conflict”	28
5.2 Rješenje: Modularnost i konvencija vlasništva	28
1. Modularnost (Knjiga od Lego kockica)	28
2. Vlasništvo (Jedna osoba = Jedna datoteka)	28
5.3 Praktičan primjer: Tijek suradnje i zaduženja	29
Tablica podjele rada	29
5.4 Oprez: Što je index.md i tko je njegov vlasnik?	30
5.5 Zlatna pravila za miran san (Bez konflikata!)	30
5.6 Test – Simulacija dodjele vlasništva	31
✓ Što smo naučili u ovom poglavlju.....	31
Poglavlje 6 - Branching: radna grana	31
6.1 Što je branch i zašto ga koristimo?	32
Analogija.....	32
Zašto ne pisati direktno na main?	32
6.2 Vizualni prikaz grananja	32
Životni ciklus jednog brancha	32
6.3 Pravila imenovanja brancheva	33
Primjeri.....	33
Pravila formata.....	33
6.4 Kreiranje brancha u Obsidianu	33
Korak 1: Otvori Command Palette.....	33
Korak 2: Kreiraj novi branch.....	33

Korak 3: Upiši naziv brancha.....	33
6.5 Prebacivanje između postojećih brancheva.....	34
6.6 Povratak na main.....	34
6.7 Git CLI alternativa.....	34
Kreiranje i prebacivanje na novi branch	35
Pregled svih brancheva	35
Prebacivanje na postojeći branch.....	35
Slanje novog brancha na GitHub.....	35
6.8 Pregled: naredbe na jednom mjestu.....	35
✅ Što smo naučili u ovom poglavlju.....	36
Poglavlje 7 - Pisanje dokumenta	36
7.1 Markdown – format u kojemu pišemo.....	36
Što je Markdown?	36
Kako to izgleda u praksi.....	36
Uvod projekta	37
7.2 Markdown osnove – sve što trebaš znati.....	37
Naslovi	37
Isticanje teksta.....	37
Nabrajanje – liste.....	38
Tablice.....	38
Linkovi i slike.....	38
Blokovi koda i napomene	39
Quarto specifične napomene (callouts)	39
Matematičke formule i izrazi.....	39
7.3 Struktura .qmd datoteke	41
YAML zaglavlje (metadata)	41
Kompletna struktura datoteke.....	41
7.4 Korištenje predloška iz repozitorija	42
Kako koristiti predložak.....	42
7.5 Dva načina prikaza u Obsidianu.....	42
Editing View (uređivanje)	42
Reading View (čitanje).....	42
7.6 Preporučeni tijek pisanja.....	43

7.7 Commit i push kroz Obsidian plugin.....	43
Korak po korak	43
Savjeti za dobre commit poruke.....	44
7.8 Git CLI alternativa.....	44
✅ Što smo naučili u ovom poglavlju.....	45
Poglavlje 8 - Pull Request: predaja rada na pregled	45
8.1 Što je Pull Request i zašto je važan?	45
Definicija	45
Zašto ne spajamo direktno?	45
Analogija.....	46
8.2 Uloge u procesu pregleda	46
8.3 Kada otvoriti Pull Request?	46
8.4 Otvaranje Pull Requesta na GitHubu	46
Korak 1: Idi na GitHub repozitorij	46
Korak 2: GitHub primijeti tvoj branch.....	47
Korak 3: Ispuni obrazac Pull Requesta	47
Korak 4: Pošalji Pull Request	48
8.5 Što se događa nakon otvaranja PR-a?	48
8.6 Kako reagirati na komentare recenzenta	49
Gdje vidiš komentare	49
Kako ispraviti prema komentarima	49
Bonton PR komunikacije	49
8.7 Provjera prije nego pushaš PR na recenziju.....	49
✅ Što smo naučili u ovom poglavlju.....	50
Poglavlje 9 - Merge u main: spajanje promjena	50
9.1 Što znači merge i kada se radi?	50
Kada je pravo vrijeme za merge?	50
9.2 Tko ima pravo mergati?	51
9.3 Merge kroz GitHub sučelje	51
Korak 1: Otvori Pull Request	51
Korak 2: Provjeri status	51
Korak 3: Odaberi tip mergea	51
Korak 4: Potvrdi merge	52

Korak 5: Obriši branch	52
9.4 Ažuriranje lokalnog repozitorija nakon mergea.....	52
U Obsidianu.....	52
Git CLI alternativa.....	53
9.5 Rješavanje merge konflikata.....	53
Što je konflikt?.....	53
Kako riješiti konflikt — korak po korak	53
Kako spriječiti konflikte	54
9.6 Nakon mergea — što dalje?.....	54
9.7 Git CLI alternativa za merge.....	54
✓ Što smo naučili u ovom poglavlju.....	55
Poglavlje 10 - Word DOCX template: koncept.....	55
10.1 Problem koji template rješava	55
10.2 Kako funkcionira reference-doc mehanizam.....	56
10.3 Gdje se template nalazi.....	56
10.4 Koje stilove template definira	56
10.5 Kako modificirati template	57
Korak 1: Otvori template u Wordu	57
Korak 2: Uredi stilove, ne sadržaj	57
Korak 3: Spremi i commitaj	57
10.6 Što se dogodi ako template nedostaje.....	58
10.7 Veza između template i finalnog dokumenta — sažetak.....	58
✓ Što smo naučili u ovom poglavlju.....	58
Poglavlje 11 - GitHub Actions: pregled automatizacije	59
11.1 Što su GitHub Actions?	59
Analogija.....	59
Što Actions radi u ovom projektu	59
11.2 Gdje se nalazi konfiguracija	60
Kako izgleda workflow datoteka (samo za razumijevanje)	60
11.3 Praćenje statusa Actions — gdje gledati	61
Zelena kvačica / Crveni X na GitHubu.....	61
Kako otvoriti Actions log.....	61
11.4 Čitanje loga — što tražiti	61

Uspješan log	61
Log s greškom	62
Kako čitati poruku greške	62
11.5 Najčešće greške i kako ih prepoznati.....	62
11.6 Preuzimanje generiranih dokumenata	63
Kako preuzeti dokument	63
11.7 Što korisnik treba znati, a što ne	63
Trebaš znati	63
Ne trebaš znati (to je posao maintainera)	63
✓ Što smo naučili u ovom poglavlju.....	64
Poglavlje 12 - Quarto render: lokalni pregled dokumenta.....	64
12.1 Zašto renderirati lokalno?.....	64
12.2 Otvaranje terminala u folderu repozitorija	65
Windows	65
macOS	65
12.3 Osnovne Quarto naredbe	65
Render jedne datoteke.....	65
Render jedne datoteke u specifičan format	65
Render cijelog projekta	66
Preview — pregled u pregledniku	66
12.4 Gdje se nalaze generirani dokumenti	66
12.5 Razumijevanje <code>_quarto.yml</code> — osnove	67
Što iz ovoga trebaš znati.....	67
12.6 Razlika između lokalnog rendera i GitHub Actions.....	67
12.7 Tipičan tijek rada s lokalnim renderom	68
12.8 Najčešće greške pri lokalnom renderu	69
✓ Što smo naučili u ovom poglavlju.....	69
Poglavlje 13 - Dodaci: Cheat Sheet, Rjecnik i Rjesavanje gresaka	70
13.1 Brzi referentni karton (Cheat Sheet)	70
13.2 Rječnik pojmova (Što znači ta riječ?)	70
13.3 Top 10 najčešćih grešaka (Troubleshooting)	71
1. “Nema mog novog poglavlja u Wordu!”	71
2. Slike se ne prikazuju (crveni križić u Quarto logu)	72

3. “GitHub ne prihvaća moju lozinku” (Authentication failed).....	72
4. “Ovaj dokument ima konflikte!” (Merge Conflict)	72
5. Gubitak Word formatiranja	72
6. Kolege ne vide moje promjene.....	72
7. “Ne mogu napraviti Push na main!”	72
8. Quarto javlja “Invalid YAML” grešku	73
9. Obsidian Git panel je nestao.....	73
10. Ne razumijem zašto se Quarto ruši	73
13.4 Korištenje AI alata (Tvoj osobni Git asistent).....	73

Priručnik: Docs-as-Code - radni tok

Kompletna dokumentacija projekta

Predgovor

Dobrodošli u službeni priručnik. Ova knjiga je generirana automatski iz Markdown datoteka.

Poglavlje 1 - Uvod: Nova paradigma rada s dokumentacijom

Preduvjeti za cijeli priručnik

Prije nego počneš, provjeri imaš li:

- Računalo s pristupom internetu
- Administratorska prava na računalu (za instalaciju softvera)
- E-mail adresu (za kreiranje GitHub računa)
- Oko **2–3 sata** za postavljanje kompletnog okruženja

Alati koje ćemo instalirati

Alat	Verzija	Namjena
Git	2.44+	Praćenje promjena i suradnja
Obsidian	1.5+	Editor za pisanje dokumentacije

Alat	Verzija	Namjena
Quarto	1.4+	Pretvorba teksta u Word/PDF/HTML
GitHub račun	—	Pohrana i dijeljenje dokumentacije

🕒 **Procijenjeno vrijeme postavljanja:** 2–3 sata (jednokratno) 🕒 **Procijenjeno vrijeme učenja toka rada:** 1 dan aktivne prakse

1.1 Stari način rada – i zašto više ne funkcionira

Zamisli ovaj scenarij koji se događa u svakom timu koji piše dokumentaciju:

1. Ana napiše prvu verziju projektnog izvješća u Wordu i pošalje ga e-mailom
2. Pero napravi izmjene i pošalje `izvjesce_v2_FINAL.docx`
3. Marija doda komentare i pošalje `izvjesce_v2_FINAL_komentari_MJ.docx`
4. Ana integrira sve izmjene i šalje `izvjesce_v3_FINAL_zapravo_final.docx`
5. Tjedan dana kasnije, nitko ne zna koja je verzija ispravna

Ovo nije greška pojedinca — **ovo je greška alata i procesa.**

Problemi starog načina rada:

- Nema jasne evidencije tko je što promijenio i kada
 - Verzije se gube, miješaju ili dupliraju
 - Paralelni rad na istom dokumentu stvara konflikte
 - Nema automatske provjere konzistentnosti formatiranja
 - Svaki put kada se mijenja sadržaj, netko mora ručno renovirati cijeli dokument
-

1.2 Novi način rada – Docs-as-Code

Docs-as-code (hrv. *dokumentacija kao kôd*) je pristup u kojem dokumentaciju pišemo, pratimo i objavljujemo **istim alatima i procesima kojima programeri prate softverski kôd.**

Umjesto Word datoteka i e-maila, koristimo:

- **Markdown** – jednostavan format za pisanje teksta
- **Git** – sustav koji pamti svaku promjenu, tko ju je napravio i kada
- **GitHub** – online pohrana i suradnički prostor
- **Obsidian** – ugodan editor za pisanje u Markdownu
- **Quarto** – alat koji pretvara Markdown u lijepo formatiran Word/PDF/HTML

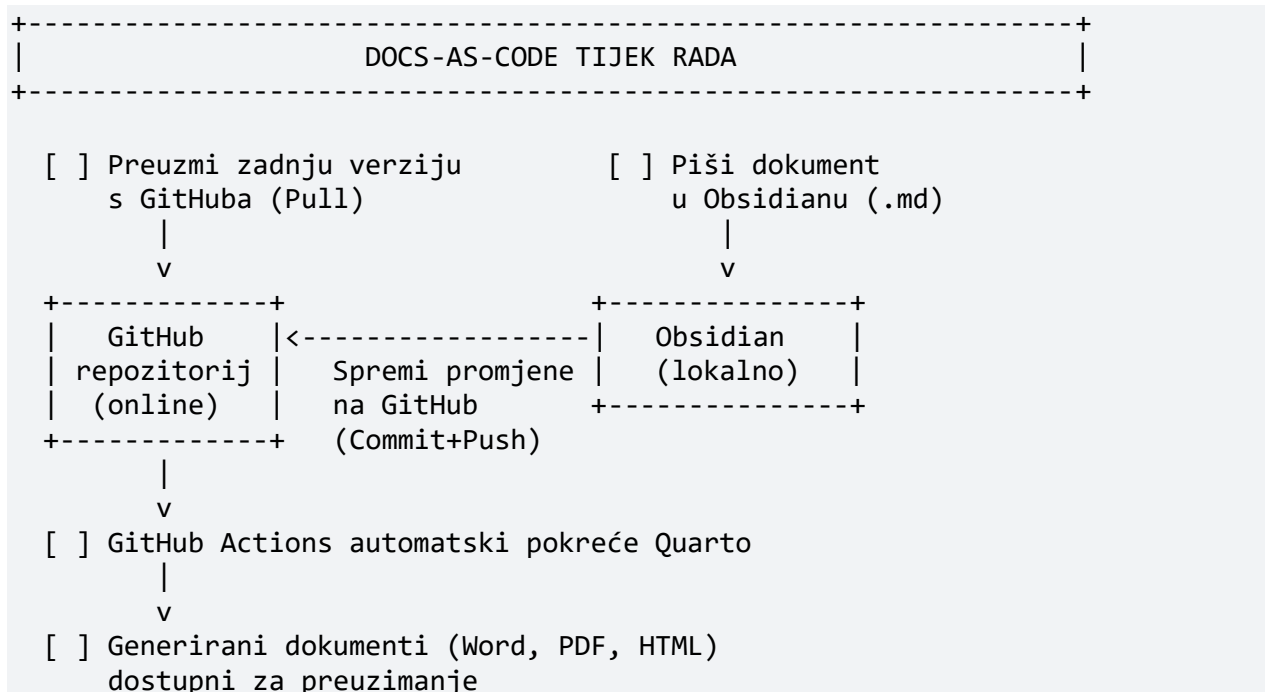
Ključna razlika: **sadržaj je odvojen od oblika.**

Ti pišeš samo tekst. Oblikovanje (fontovi, stilovi, numeracija) se primjenjuje automatski i konzistentno pri svakom generiranju dokumenta.

1.3 Usporedba starog i novog načina

Značajka	Stari način (Word + e-mail)	Novi način (Docs-as-code)
Verzioniranje	Ručno (_v2_FINAL)	Automatsko, svaka promjena zabilježena
Suradnja	Slanje datoteka e-mailom	Paralelni rad, bez e-maila
Praćenje promjena	Komentari u Wordu	Potpuna povijest tko je što promijenio
Formatiranje	Svaki put iznova	Jednom definirano, uvijek konzistentno
Objavljivanje	Ručno generiranje PDF-a	Automatski pri svakom ažuriranju
Pohrana	Lokalno + e-mail privitak	Centralna pohrana, dostupno svima
Greška	“Koja je verzija ispravna?”	Uvijek jasno koja je zadnja verzija

1.4 Tijek rada – kako to izgleda u praksi



Korak po korak u svakodnevnom radu:

1. **Pull** — Jutarnji ritual: preuzmi sve izmjene koje su napravili kolege
 2. **Branch** — Kreiraj svoju “radnu granu” (vlastitu kopiju projekta)
 3. **Piši** — Piši u Obsidianu kao u bilo kojem editoru teksta
 4. **Commit** — Spremi promjenu s kratkim opisom što si napravio/la
 5. **Push** — Pošalji promjene na GitHub
 6. **Pull Request** — Predaj rad na pregled kolegi
 7. **Merge** — Nakon pregleda, promjene se spajaju s glavnom verzijom
 8. **Automatski render** — GitHub automatski generira finalni dokument
-

1.5 Uloga svakog alata

Git

Što je: Sustav za praćenje promjena u datotekama.

Analogija: Zamisli Git kao napredni “Track Changes” u Wordu, ali koji radi za sve datoteke odjednom i pamti **kompletanu povijest** svih promjena od samog početka projekta.

Što radi za tebe: - Pamti tko je što promijenio i kada - Omogućava vraćanje na bilo koju prethodnu verziju - Omogućava paralelni rad više osoba bez konflikata

Gdje živi: Na tvom računalu (lokalno)

GitHub

Što je: Online platforma koja pohranjuje Git repozitorije.

Analogija: GitHub je kao Google Drive, ali za kod i dokumentaciju — s ugrađenim sustavom pregleda, komentiranja i odobravanja promjena.

Što radi za tebe: - Centralna pohrana svih datoteka projekta - Pregled i odobravanje tuđih izmjena (Pull Request) - Automatsko pokretanje alata (GitHub Actions) - Evidencija svih promjena s imenima autora

Gdje živi: Online, na github.com

Obsidian

Što je: Editor za pisanje tekstualnih datoteka u Markdown formatu.

Analogija: Obsidian je kao Word, ali piše u jednostavnom tekstualnom formatu koji Git može pratiti. Nema skrivenih XML datoteka — samo čisti tekst.

Što radi za tebe: - Ugodno sučelje za pisanje dokumentacije - Pregled Markdown teksta (kako će izgledati) - Git integracija kroz plugin (commit, push — bez terminala) - Brza navigacija između datoteka projekta

Gdje živi: Na tvom računalu (lokalno)

Quarto

Što je: Alat koji pretvara Markdown datoteke u finalne dokumente.

Analogija: Quarto je kao printer koji zna i oblikovati dokument. Ti daš čisti tekst, Quarto doda profesionalne stilove i izveze Word, PDF ili web stranicu.

Što radi za tebe: - Pretvara .qmd i .md datoteke u .docx, .pdf, .html - Primjenjuje konzistentne stilove (iz Word predloška) - Automatski numerira naslove, slike, tablice - Može se pokrenuti ručno ili automatski (GitHub Actions)

Gdje živi: Na tvom računalu + automatski na GitHubu

1.6 Zašto je ovo bolje? – Konkretno prednosti

Za tebe kao autora: - Fokusiraš se na sadržaj, ne na formatiranje - Znaš točno što si promijenio/la i kada - Možeš se uvijek vratiti na stariju verziju ako zatreba - Nema straha od “pokvario/la sam dokument”

Za tim: - Svatko zna na čemu radi tko - Nema konflikata između verzija - Svaka promjena je vidljiva i objašnjena - Pregled rada kolega je strukturiran i dokumentiran

Za projekt: - Kompletan revizijska staza — uvijek se zna što se promijenilo i zašto - Automatsko generiranje finalnih dokumenata - Konzistentno formatiranje u svim dokumentima

1.7 Što ćeš naučiti u ovom priručniku

Do kraja priručnika, moći ćeš **samostalno:**

- Postaviti kompletan radni alat na svom računalu
- Klonirati projektni repozitorij i otvoriti ga u Obsidianu
- Pisati dokumentaciju u Markdown formatu
- Raditi na vlastitoj grani bez ometanja kolega

- Predati rad na pregled putem Pull Requesta
- Generirati finalni Word dokument pomoću Quartoa

☑ Što smo naučili u ovom poglavlju

Koncept	Objašnjenje
Docs-as-code	Pristup pisanja dokumentacije istim alatima kao softverski kôd
Git	Lokalni sustav za praćenje svih promjena u datotekama
GitHub	Online platforma za pohranu i suradnju
Obsidian	Editor za pisanje u Markdown formatu
Quarto	Alat za pretvorbu teksta u Word/PDF/HTML
Tijek rada	Pull -> Branch -> Piši -> Commit -> Push -> PR -> Merge

💡 **Savjet:** Ne moraš odmah razumjeti svaki pojam. Svaki korak bit će objašnjen konkretnim primjerima u sljedećim poglavljima. Sada je dovoljno da imaš opću sliku kako dijelovi funkcioniraju zajedno.

Sljedeće poglavlje: [Poglavlje 2 – Instalacija i postavljanje okruženja ->](#)

Poglavlje 2 - Instalacija i postavljanje okruženja

🕒 **Procijenjeno vrijeme:** 60–90 minuta (jednokratno) 🖥️ **Primarno za:** Windows
10/11 🍏 **Korisnici macOS-a:** Tamo gdje se postupak razlikuje, označeno je posebnom napomenom.

U ovom poglavlju instaliramo sve alate i kreiramo račune koje ćeš koristiti svaki dan. Ovo radiš **samo jednom** — nakon postavljanja, svakodnevni rad ne zahtijeva ponovnu instalaciju.

2.1 Git – instalacija

Što je Git i zašto ga instaliramo?

Git je program koji radi u pozadini i **prati sve promjene** u tvojim datotekama. Obsidian Git plugin (kojeg instaliramo u Poglavlju 3) koristi Git ispod haube — bez njega plugin ne može raditi.

Korak 1: Preuzimanje

1. Otvori preglednik i idi na: **<https://git-scm.com/download/win>**
2. Preuzimanje će se automatski pokrenuti za tvoj Windows
3. Sačekaj da se preuzme datoteka (npr. Git-2.44.0-64-bit.exe)

macOS: Idi na <https://git-scm.com/download/mac> i slijedi upute za instalaciju putem Homebrew-a, ili jednostavno otvori Terminal i upiši `git --version` — macOS će ponuditi automatsku instalaciju.

Korak 2: Instalacija (Windows)

1. Pokreni preuzetu .exe datoteku dvostrukim klikom
2. Prihvati sve zadane opcije klikom na **Next** kroz cijeli instalacijski čarobnjak
3. Na ekranu **“Choosing the default editor used by Git”** — ostavi zadanu opciju (Vim) ili odaberi **Notepad** ako se osjećaš nelagodno s Vimom. Za naš rad, ovo nije važno jer editor koristimo samo u Obsidianu.
4. Na ekranu **“Adjusting your PATH environment”** — obavezno odaberi **“Git from the command line and also from 3rd-party software”** (srednja opcija). Ovo omogućava Obsidianu da pronađe Git.
5. Sve ostalo — klikaj **Next** i na kraju **Install**

⚠ **Upozorenje:** Ako instalacijski čarobnjak pita za administratorsku lozinku, upiši ju — instalacija zahtijeva administratorska prava.

Korak 3: Provjera instalacije

1. Pritisni tipke **Windows + R** na tipkovnici
2. Upiši `cmd` i pritisni Enter — otvara se crni prozor (Command Prompt)
3. Upiši sljedeću naredbu i pritisni Enter:

```
git --version
```

✅ **Provjera uspjeha:** Trebao/la bi vidjeti nešto poput:

```
git version 2.44.0.windows.1
```

⚠ **Ako vidiš grešku** `'git' is not recognized...` — Git nije ispravno instaliran. Ponovi instalaciju i provjeri jesi li na koraku 4 odabrao/la srednju opciju za PATH.

Korak 4: Konfiguracija identiteta

Git treba znati tko si — ovo ime i e-mail će se prikazivati uz svaku promjenu koju napraviš.

U istom crnom prozoru (Command Prompt) upiši ove dvije naredbe, **zamijeni tekst u navodnicima** s tvojim stvarnim imenom i e-mailom:

```
git config --global user.name "Ime Prezime"
```

```
git config --global user.email "tvoj.email@primjer.com"
```

💡 **Savjet:** Koristi **isti e-mail koji ćeš koristiti za GitHub račun** (kreiramo ga u koraku 2.4). To pomaže GitHubu da ispravno poveže tvoje doprinose s tvojim profilom.

✅ **Provjera:** Provjeri jesu li podaci ispravno uneseni:

```
git config --global user.name  
git config --global user.email
```

Svaka naredba ispisat će vrijednost koju si upravo postavio/la.

2.2 Obsidian – instalacija

Što je Obsidian?

Obsidian je editor — program za pisanje teksta. Koristit ćeš ga svaki dan za pisanje dokumentacije. Izgleda kao Word, ali sprema datoteke u jednostavan tekstualni format koji Git može pratiti.

Korak 1: Preuzimanje

1. Idi na: **<https://obsidian.md>**
2. Klikni na veliki gumb **“Download”**
3. Odaberi verziju za **Windows** (.exe instalacijska datoteka)
macOS: Odaberi .dmg verziju i instaliraj kao i svaki drugi Mac program (odvuci u Applications).

Korak 2: Instalacija

1. Pokreni preuzetu .exe datoteku
2. Obsidian se instalira automatski — nema čarobnjaka s opcijama
3. Obsidian će se automatski pokrenuti nakon instalacije

Korak 3: Početni ekran

Kada se Obsidian prvi put pokrene, prikazuje se ekran dobrodošlice s opcijama. **Nemoj još ništa birati** — vault (projektni folder) kreirat ćemo u Poglavlju 4, kada kloniramo repozitorij.

Možeš zatvoriti Obsidian za sada.

✅ **Provjera uspjeha:** Obsidian se pokrenuo i prikazuje početni ekran.

2.3 Quarto – instalacija

Što je Quarto?

Quarto je alat koji radi u pozadini — pretvara tvoje tekstualne datoteke u finalne Word dokumente. Nećeš ga koristiti direktno svaki dan, ali mora biti instaliran da bi automatizacija (GitHub Actions) i lokalni pregled funkcionirali.

Korak 1: Preuzimanje

1. Idi na: **<https://quarto.org/docs/get-started/>**
2. Klikni na gumb za preuzimanje za **Windows** (.msi datoteka)
macOS: Preuzmi .pkg verziju.

Korak 2: Instalacija

1. Pokreni preuzetu .msi datoteku
2. Prihvati sve zadane opcije i klikaj **Next** do kraja
3. Klikni **Finish**

Korak 3: Provjera instalacije

1. Zatvori i ponovo otvori Command Prompt (Windows + R -> cmd)
2. Upiši:

```
quarto check
```

✅ **Provjera uspjeha:** Nakon nekoliko sekundi vidjet ćeš popis provjera. Važno je da nema crvenih grešaka za osnovne komponente. Upozorenja (žuta) su u redu.

⚠️ **Ako naredba nije prepoznata:** Restart računala i pokušaj ponovo — instalacija Quarta ponekad zahtijeva ponovno pokretanje računala da bi se PATH ažurirao.

2.4 GitHub – kreiranje računa

Što je GitHub?

GitHub je online platforma gdje se pohranjuje sva dokumentacija projekta. Svaki član tima ima vlastiti račun. Kroz GitHub koordiniramo tko radi što i pregledavamo tuđi rad.

Korak 1: Registracija

1. Idi na: **<https://github.com>**
2. Klikni na “**Sign up**” u gornjem desnom kutu
3. Upiši svoju **e-mail adresu** (isti e-mail koji si koristio/la u Git konfiguraciji!)
4. Kreiraj **lozinku** — GitHub traži minimalnu duljinu i složenost

5. Odaberi **korisničko ime** (username) — ovo će biti vidljivo svim članovima tima.
Preporuka: ime-prezime ili varijanta
- Savjet za korisničko ime:** Odaberi nešto profesionalno jer će biti vidljivo u svim pull requestima i komentarima. Primjeri: ana-kovac, pero-novak, mj-horvat
6. Dovrši verifikaciju (puzzle ili slično)
7. GitHub će poslati **verifikacijski e-mail** — otvori ga i klikni na link za potvrdu

Korak 2: Traženje pristupa repozitoriju

Samo kreiranje računa nije dovoljno — administrator projekta mora te **dodati u tim** koji ima pristup repozitoriju.

Nakon kreiranja računa, pošalji administratoru projekta: - Svoje GitHub **korisničko ime** - Svoju **e-mail adresu** vezanu uz GitHub

Administrator će ti poslati pozivnicu na e-mail. **Klikni na link u e-mailu** da prihvatiš pozivnicu. Tek tada ćeš imati pristup repozitoriju.

2.5 Personal Access Token (PAT) – postavljanje autentikacije

Zašto nam treba PAT?

Kada Obsidian šalje tvoje promjene na GitHub, GitHub mora znati da si zaista ti — a ne netko tko se lažno predstavlja. Umjesto lozinke, GitHub koristi **Personal Access Token (PAT)** — poseban ključ koji koristimo samo za ovaj tip pristupa.

i Za razumijevanje: PAT je poput privremene propusnice — možeš ga poništiti u bilo kojem trenutku ako sumnjaš da ga je netko drugi dobio.

Korak 1: Kreiranje PAT-a na GitHubu

1. Prijavi se na **https://github.com**
2. Klikni na svoju **profilnu sliku** u gornjem desnom kutu
3. Odaberi **Settings** (Postavke)
4. U lijevom izborniku skrolaj do dna i klikni na **Developer settings**
5. Klikni na **Personal access tokens -> Tokens (classic)**
6. Klikni na gumb **“Generate new token” -> “Generate new token (classic)”**

Korak 2: Konfiguracija tokena

Na ekranu za kreiranje tokena:

- **Note** (naziv): Upiši npr. Obsidian-Git-token — da znaš čemu služi
- **Expiration** (trajanje): Odaberi **90 days** ili **1 year** ovisno o preferenciji tima

💡 **Savjet:** Kada token istekne, moraš kreirati novi i ponoviti korake 3 i 4 iz ovog poglavlja. Postavi podsjetnik u kalendar.

- **Odaberi ovlasti (scopes):** Stavi kvačicu samo na **repo** — ovo je sve što trebaš za rad s repozitorijem

7. Skrolaj do dna i klikni **“Generate token”**

Korak 3: Kopiranje i pohrana tokena

⚠️ **Važno:** Token ćeš vidjeti **samo jednom**. Kada zatvoriš ovu stranicu, više ga nećeš moći vidjeti. GitHub pokazuje token samo u trenutku kreiranja.

1. Klikni na ikonu kopiranja pored tokena (ili označi tekst i kopiraj Ctrl+C)
2. Odmah ga spremi na **sigurno mjesto** — primjeri:
 - Privatna bilješka u password manageru (preporuka)
 - Zaštićena tekstualna datoteka na računalu
 - Privremeno: nalijepi ga u Notepad dok ga ne postaviš u Obsidian

Token izgleda otprilike ovako (ovo je primjer, ne koristi ga):

```
ghp_ABCDEFGHi jklmnopQRSTUVWXYZ123456789
```

Korak 4: Pohrana tokena

Token ćeš upisati direktno u **Obsidian Git plugin** u sljedećem poglavlju — nema potrebe za dodatnim koracima ovdje.

💡 **Savjet:** Ostavi token kopiran u Notepadu ili na sigurnom mjestu dok ne završiš postavljanje plugina u Poglavlju 3 — tamo ćeš ga unijeti na točno određeno mjesto u postavkama.

✅ **Provjera uspjeha:** Token je kreiran i pohranjen na sigurnom mjestu, spreman za unos u Obsidian.

2.6 .gitignore – što dijelimo, a što ne

Što je .gitignore?

.gitignore je tekstualna datoteka koja Git-u govori koje datoteke i foldere treba **ignorirati** — tj. nikada ih ne pratiti niti slati na GitHub.

Zašto je to važno za Obsidian?

Obsidian pohranjuje svoje postavke u skriveni folder `.obsidian/`. Taj folder sadrži:

Sadržaj	Treba dijeliti?	Razlog
plugins/	✅ Da	Da svi imaju iste

Sadržaj	Treba dijeliti?	Razlog
app.json	✓ Da	plugine Zajedničke postavke aplikacije
workspace.json	✗ Ne	Osobne postavke prozora — razlikuju se po računalu
cache	✗ Ne	Privremene datoteke — automatski se kreiraju

Primjer .gitignore za ovaj projekt

Datoteka .gitignore već postoji u repozitoriju. Ovako izgleda relevantni dio za Obsidian:

```
# Obsidian - osobne postavke koje NE dijelimo
.obsidian/workspace.json
.obsidian/workspace-mobile.json
.obsidian/cache

# Obsidian - ove datoteke DIJELIMO (plugin konfiguracija)
# (nisu navedene u .gitignore, što znači da Git ih prati)
# .obsidian/plugins/
# .obsidian/app.json
# .obsidian/community-plugins.json

# Sistemske datoteke
.DS_Store          # macOS metapodaci
Thumbs.db          # Windows metapodaci
desktop.ini        # Windows postavke foldera

# Quarto output (generirani dokumenti)
/_site/
/.quarto/
```

💡 **Savjet:** Ne moraš mijenjati .gitignore datoteku. Administrator projekta već je postavio ispravne postavke u repozitoriju koji ćeš klonirati. Ovo je samo objašnjenje zašto neke datoteke vidiš, a neke ne.

☑ Što smo naučili u ovom poglavlju

Alat/Korak	Status
Git instaliran i konfiguriran s imenom i e-mailom	[]

Alat/Korak	Status
Obsidian instaliran	[]
Quarto instaliran i provjeren	[]
GitHub račun kreiran i verificiran	[]
Pozivnica za repozitorij prihvaćena	[]
PAT kreiran i sigurno spremljen	[]

💡 **Savjet:** Iskoristi gornju tablicu kao checklistu — označi svaki korak koji si dovršio/la. Ako nešto nije funkcioniralo, ne prelazi na sljedeće poglavlje — svaki alat ovisi o prethodnom.

<- Prethodno poglavlje: [Poglavlje 1 – Uvod](#) Sljedeće poglavlje: [Poglavlje 3 – Obsidian Git plugin](#) ->

Poglavlje 3 - Obsidian Git plugin: instalacija i konfiguracija

🕒 **Procijenjeno vrijeme:** 15–20 minuta 📄 **Preduvjet:** Git instaliran (Poglavlje 2.1), PAT token kreiran i negdje pohranjen (Poglavlje 2.5)

⚠️ **Važno:** Ovo poglavlje radi **bez kloniranog repozitorija** — plugin instaliramo dok Obsidian radi na privremenom vaultu. U Poglavlju 4 otvorit ćemo pravi projektni repozitorij.

3.1 Pokretanje Obsidiana i privremeni vault

Obsidian ne može raditi bez **vaulta** (hrv. *trezor*) — foldera koji je postavljen kao radni prostor. Budući da još nismo klonirali repozitorij, koristit ćemo privremeni vault samo za instalaciju plugina.

1. Pokreni **Obsidian**
2. Na početnom ekranu klikni **“Create new vault”**
3. Daj mu bilo koji naziv (npr. privremeni) i odaberi lokaciju (npr. Desktop)
4. Klikni **“Create”**

Obsidian se otvara s praznim vaultom. Sad možemo instalirati plugin.

3.2 Omogućavanje Community plugina

Obsidian dolazi s ugrađenim pluginima, ali **Obsidian Git** je *community plugin* — plugin kojeg su razvili vanjski developeri, nije dio službenog Obsidiana.

Da bi koristili community plugine, moramo ih najprije omogućiti:

1. Klikni na ikonu **zupčanika**  u donjem lijevom kutu (Settings / Postavke)
2. U lijevom izborniku klikni na **“Community plugins”**
3. Vidjet ćeš upozorenje: *“Community plugins can execute arbitrary code...”*

i **O sigurnosnom upozorenju:** Ovo upozorenje postoji jer community plugine pišu vanjski developeri. **Obsidian Git** je jedan od najpopularnijih i najprovjerenijih plugina s desetcima tisuća korisnika i otvorenim izvornim kodom. Prihvati upozorenje.

4. Klikni na **“Turn on community plugins”**
5. Potvrdi klikom na **“Turn on”**

3.3 Instalacija Obsidian Git plugina

1. U istom ekranu (Community plugins), klikni na gumb **“Browse”** (Pregledaj)
2. U polje za pretraživanje upiši: Obsidian Git
3. U rezultatima klikni na **“Obsidian Git”** (autor: Vinzent03)
4. Klikni na gumb **“Install”**
5. Nakon instalacije klikni na **“Enable”** (Omogući)

✓ Provjera uspjeha: U lijevom izborniku Settings-a trebala bi se pojaviti nova stavka **“Obsidian Git”** pod sekcijom *Plugin Options*.

3.4 Konfiguracija plugina – autentikacija

Ovo je najvažniji korak — ovdje unosimo naš PAT token da Obsidian može komunicirati s GitHubom.

1. U Settings izborniku klikni na **“Obsidian Git”**
2. Skrolaj do sekcije **“Authentication/Commit Author”**
3. Ispuni sljedeća polja:

Polje	Što upisati	Primjer
Username	Tvoje GitHub korisničko ime	ana-kovac
Password/Token	PAT token koji si kreirao/la u Poglavlju 2.5	ghp_ABC...
Author name	Tvoje ime i prezime	Ana Kovač
Author email	E-mail vezan uz GitHub račun	ana@primjer.com

💡 **Savjet:** Polje za token izgleda kao obično tekstualno polje — token upisuješ ili lijepiš (Ctrl+V) direktno u njega. Ne skrivaj ga u lozinkovnom manageru sada — Obsidian ga sam sigurno pohranjuje.

⚠️ **Upozorenje:** Author name i Author email moraju biti **isti** kao oni koje si postavio/la u Git konfiguraciji (Poglavlje 2.1, Korak 4). Inače će tvoje promjene biti pripisane drugom identitetu.

3.5 Konfiguracija plugina – automatizacija

Ostale postavke koje preporučujemo za svakodnevni rad:

Automatski pull

Skrolaj do sekcije “**Automatic**”:

Postavka	Preporučena vrijednost	Objašnjenje
Pull updates on startup	Uključeno	Svaki put kad otvoriš Obsidian, automatski preuzme najnovije promjene kolega
Pull interval (minutes)	0 (isključeno)	Automatski pull svakih N minuta — za početnike isključi, radi ručno
Push on commit	Uključeno	Nakon svakog commita, automatski šalje promjene na GitHub

💡 **Zašto isključiti automatski pull interval?** Automatski pull može prekinuti pisanje u neočekivanom trenutku. Bolja navika za početnike je: ručno povuci promjene ujutro kada sjedneš raditi.

Commit poruke

Skrolaj do sekcije “**Commit**”:

Postavka	Preporučena vrijednost
Commit message	{date} - {{hostname}}: {{numFiles}} datoteka izmijenjeno
Date format	YYYY-MM-DD HH:mm

💡 **Objašnjenje commit poruke:** {date} automatski upiše datum i vrijeme, {hostname} ime tvog računala, {numFiles} broj izmijenjenih datoteka. Rezultat izgleda ovako: 2024-03-15 09:30 - ANA-LAPTOP: 2 datoteke izmijenjeno.

3.6 Pregled sučelja plugina

Nakon konfiguracije, plugin dodaje nekoliko elemenata u Obsidian sučelje:

Source Control panel (bočna traka)

Klikni na ikonu **grananja** (<) u desnoj bočnoj traci (ili pritisni Ctrl+Shift+G).

Otvora se **Source Control** panel s:

SOURCE CONTROL			R ^
Changes (Promjene)			
-- dokument1.md	[M]		<- M = Modified
-- novi-dokument.md	[U]		<- U = Untracked
-- obrisan.md	[D]		<- D = Deleted
[Stage All]	[Commit]	[Push]	

Gumbi koje ćeš koristiti svaki dan

Gumb	Što radi
v Pull	Preuzima najnovije promjene s GitHuba
Stage All	Označava sve promjene kao “spremne za commit”
Commit	Sprema promjene s opisnom porukom
^ Push	Šalje commitove na GitHub

Status bar (statusna traka)

Na dnu Obsidian prozora vidjet ćeš informacije o trenutnom stanju:

Current branch: main | v0 ^2

Što znači: - **Current branch**: na kojoj si grani (više o granama u Poglavlju 6) - **v0** — nema novih promjena za preuzeti s GitHuba - **^2** — imaš 2 commita koja još nisu poslana na GitHub

3.7 Što plugin radi automatski, a što ručno

Radnja	Automatski	Ručno
Pull pri pokretanju Obsidiana	(ako je uključeno)	Gumb v Pull
Praćenje izmjena datoteka	uvijek	—
Stage promjena	Ne	Gumb “Stage All”

Radnja	Automatski	Ručno
Commit	Ne	Gumb "Commit"
Push na GitHub	(nakon commita, ako je uključeno)	Gumb ^ Push
Kreiranje novog brancha	Ne	Command Palette (objašnjeno u Poglavljju 6)

💡 **Preporučeni dnevni ritam:** 1. Otvori Obsidian -> automatski pull preuzima novosti 2. Piši dokument 3. Kada završiš logičnu cjelinu -> Stage All -> Commit 4. Na kraju radnog dana (ili češće) -> Push

☑ Što smo naučili u ovom poglavljju

Koncept	Objašnjenje
Community plugin	Plugin razvijen od strane zajednice, nije dio službenog Obsidiana
Autentikacija	Obsidian Git koristi PAT token za slanje promjena na GitHub
Source Control panel	Pregled izmijenjenih datoteka i gumbi za commit/push
Push on commit	Automatsko slanje na GitHub nakon svakog commita
Pull on startup	Automatsko preuzimanje izmjena kolega pri pokretanju Obsidiana

📘 **Napomena:** Plugin si konfigurirao/la u privremenom vaultu. Kada u Poglavljju 4 otvorimo pravi repozitorij kao novi vault, **morat ćeš ponoviti instalaciju i konfiguraciju plugina** — postavke se pohranjuju po vaultu, ne globalno. Proces je isti, a ovaj put znaš točno što radiš.

<- **Prethodno poglavlje:** [Poglavlje 2 – Instalacija](#) **Sljedeće poglavlje:** [Poglavlje 4 – Kloniranje repozitorija i kreiranje Obsidian vaulta](#) ->

Poglavlje 4 - Kloniranje repozitorija i kreiranje Obsidian vaulta

Procijenjeno vrijeme: 10–15 minuta

Cilj: Preuzeti projekt s GitHuba na računalo i otvoriti ga u Obsidianu.

Sada kada su svi alati instalirani, vrijeme je da na svoje računalo preuzmемо stvarni projekt (dokumentaciju) na kojem ćemo raditi. Ovaj proces nazivamo **kloniranje**.

4.1 Što je repozitorij i što znači “kloniranje”?

Prije nego što upišemo naredbu, važno je razumjeti što točno radimo.

Repozitorij (ili skraćeno “repo”) je glavni direktorij (folder) tvog projekta koji se nalazi na internetu (na GitHubu). On sadrži sve datoteke, slike i kompletnu povijest svih promjena. Zamisli ga kao glavni, zajednički folder na Google Driveu.

Kloniranje (Cloning) nije obično preuzimanje (Download). Kada “skidaš” običnu datoteku s interneta, ona gubi vezu sa svojim izvorom. Kada **kloniraš** Git repozitorij, ti na svoje računalo preuzimaš identičnu, radnu kopiju koja trajno pamti odakle je došla i ostaje povezana s glavnim repozitorijem na GitHubu. To nam omogućuje kasnije slanje i preuzimanje novih promjena.

4.2 Kloniranje repozitorija na računalo

Sada ćemo klonirati testni repozitorij na tvoju Radnu površinu (Desktop).

Korak po korak:

1. Otvori **Command Prompt** (Pritisni tipku Windows + R, upiši cmd i pritisni Enter).

macOS: Otvori aplikaciju **Terminal**.

2. Prvo moramo reći terminalu u koji folder želimo spremiti projekt. Upiši sljedeću naredbu i pritisni Enter:
cd Desktop

(Ova naredba znači “Change Directory to Desktop” - prebaci me na Radnu površinu).

3. Sada upiši naredbu za kloniranje (ili je kopiraj i zalijepi) i pritisni Enter:
git clone https://github.com/FGAG-docs/EU-Project-Template

4. **Što se događa sada?**

Vidjet ćeš tekst koji ispisuje postotke (Receiving objects: 100%...). Git preuzima sve datoteke i cijelu povijest projekta s GitHuba na tvoje računalo.

i Napomena o prijavi: Ako repozitorij nije javno dostupan, u ovom trenutku iskočit će prozor ili će terminal tražiti tvoje GitHub korisničko ime i lozinku. Kao lozinku **obavezno upiši onaj PAT token** koji si kreirao/la i spremio/la u Poglavlju 2! (Ako si u Poglavlju 2 uspješno podesio/la Credential Manager, ovaj korak će se možda odraditi potpuno automatski).

✅ **Provjera uspjeha:** Smanji sve prozore i pogledaj svoju Radnu površinu. Tamo bi se sada trebao nalaziti novi folder pod nazivom EU-Project-Template.

4.3 Otvaranje kloniranog foldera kao Obsidian “Vault”

Sada imamo datoteke na računalu, ali ih želimo uređivati u našem pametnom editoru. U Obsidianu se svaki projektni folder naziva **Vault** (trezor).

Korak po korak:

1. Otvori program **Obsidian**.
2. Ako ti se otvori onaj probni Vault iz Poglavlja 3, klikni na ikonu “Vault” u donjem lijevom kutu (izgleda kao sef s otvorenim vratima) kako bi otvorio/la glavni izbornik.
3. U glavnom izborniku Obsidiana, pored opcije “**Open folder as vault**” (Otvori mapu kao trezor), klikni na gumb **Open**.
4. Otvorit će se prozor za odabir foldera. Navigiraj na svoju Radnu površinu (Desktop), klikni na folder EU-Project-Template i pritisni **Select Folder** (ili Odaberi mapu).
5. Ako te Obsidian pita “Do you trust the authors of the files in this folder?” (Vjeruješ li autorima...), klikni “**Trust author and enable plugins**”. Ovo je važno jer omogućava rad našem Obsidian Git pluginu.

✅ **Provjera uspjeha:** S lijeve strane Obsidian ekrana sada vidiš popis datoteka i foldera tvog projekta.

4.4 Razumijevanje strukture repozitorija

Kada pogledaš lijevi izbornik u Obsidianu, vidjet ćeš razne foldere i datoteke. Ovdje je objašnjenje što oni znače i koje smiješ dirati:

Struktura repozitorija

Ime foldera/datoteke	Namjena	Što ti radiš s tim?
.obsidian	Ovdje Obsidian čuva svoje postavke i plugine.	Ne diraj. Git je uglavnom podešen da ignorira tvoje lokalne promjene ovdje.
.github	Ovdje se nalaze upute za “robote” (GitHub Actions) koji automatski generiraju tvoj Word/PDF dokument.	Ne diraj. Ovo je posao administratora sustava.
Slike (ili images)	Folder u koji ubacujemo sve vizualne materijale.	Koristiš. Tu spremaš slike koje želiš umetnuti u dokument.

Ime foldera/datoteke	Namjena	Što ti radiš s tim?
*.md ili *.qmd datoteke	Ovo su tvoji stvarni tekstualni dokumenti (npr. 01-Uvod.md, 02-Metodologija.qmd).	Ovdje pišeš. Ovo su datoteke na kojima ćeš raditi svaki dan.
_quarto.yml	Konfiguracijska datoteka koja govori Quartu kako da spoji sve Markdown datoteke u jednu Word knjigu.	Oprezno. Mijenjaš samo kada dodaješ novo poglavlje u strukturu knjige.
reference-doc.docx	Prazan Word dokument koji služi kao vizualni predložak (definira fontove, veličine naslova, boje).	Ne diraj osim ako nisi zadužen/a za vizualni identitet dokumenta.

Savjet: Markdown datoteke obično imaju nastavak .md, a u našem sustavu mogu imati i nastavak .qmd (Quarto Markdown). Za tebe kao pisca nema nikakve razlike – obje vrste otvaraš i uređuješ u Obsidianu na potpuno identičan način.

☒ Što smo naučili u ovom poglavlju

Koncept / Korak	Status
Razumijem razliku između običnog “Downloada” i “Kloniranja”	[]
Znam pozicionirati terminal na Desktop pomoću komande cd Desktop	[]
Uspješno sam pokrenuo/la naredbu git clone ...	[]
Novi folder EU-Project-Template nalazi se na mom računalu	[]
Projekt je uspješno otvoren u Obsidianu (Open folder as vault)	[]
Znam u kojim folderima pišem, a koje sistemske foldere ne diram	[]

Poglavljje 5 - Podjela rada i timska suradnja

 **Procijenjeno vrijeme čitanja:** 10–15 minuta  **Preduvjet:** Repozitorij kloniran i vault otvoren (Poglavljje 4)

5.1 Noćna mora zvana “Merge Conflict”

U klasičnom načinu rada (Word datoteka na dijeljenom disku ili slanje mailom), najveći strah tima je “pregaziti” tuđi rad. U Git svijetu, tvoj rad je uvijek siguran jer radiš u izoliranom okruženju. Međutim, problem može nastati u trenutku spajanja (Merge).

Zamisli ovu situaciju: 1. Ti i kolega radite na istoj datoteci (01-uvod.md). 2. Ti izmijeniš prvu rečenicu u “*Ovaj projekt je izvrstan.*” 3. Kolega izmijeni **potpuno istu rečenicu** u “*Ovaj projekt je sjajan.*” 4. Oboje pošaljete promjene na GitHub.

Kada pokušate spojiti te dvije grane, Git će stati, podići “ručnu kočnicu” i reći: “*Čekaj malo! Imam dvije različite verzije iste rečenice. Ne znam koja je točna. Ljudi, riješite ovo sami.*”

To se zove **Merge Conflict (Konflikt pri spajanju)**. Sustav će u dokument ubaciti čudne oznake (<<<<<<, =====) i tražiti od tebe da ručno odabereš verziju koju želiš zadržati.

Utješna činjenica: Konflikt **nije** katastrofa! Git ne briše podatke, ništa nije izgubljeno i nikome nisi obrisao/la tekst. Sustav samo traži ljudsku odluku. Ipak, za početnike su konflikti stresni, pa ih želimo izbjeći u potpunosti.

5.2 Rješenje: Modularnost i konvencija vlasništva

Kako bismo u startu eliminirali 99% mogućnosti za konflikte, koristimo dva jednostavna principa Docs-as-Code metodologije: **Modularnost** i **Vlasništvo**.

1. Modularnost (Knjiga od Lego kockica)

U Wordu obično imamo jednu divovsku datoteku od 100 stranica. U našem repozitoriju, projekt je “razbijen” na mnogo malih datoteka (modula).

Klasični pristup:	Docs-as-Code pristup:
Glavni_Dokument_v7_final.docx	├─ 01-uvod.qmd
	├─ 02-metodologija.qmd
	├─ 03-budzet.qmd
	└─ 04-zakljucak.qmd

Naš “printer” (Quarto alat) će na kraju samostalno uzeti sve ove male datoteke i savršeno ih slijepiti u jedan veliki Word dokument.

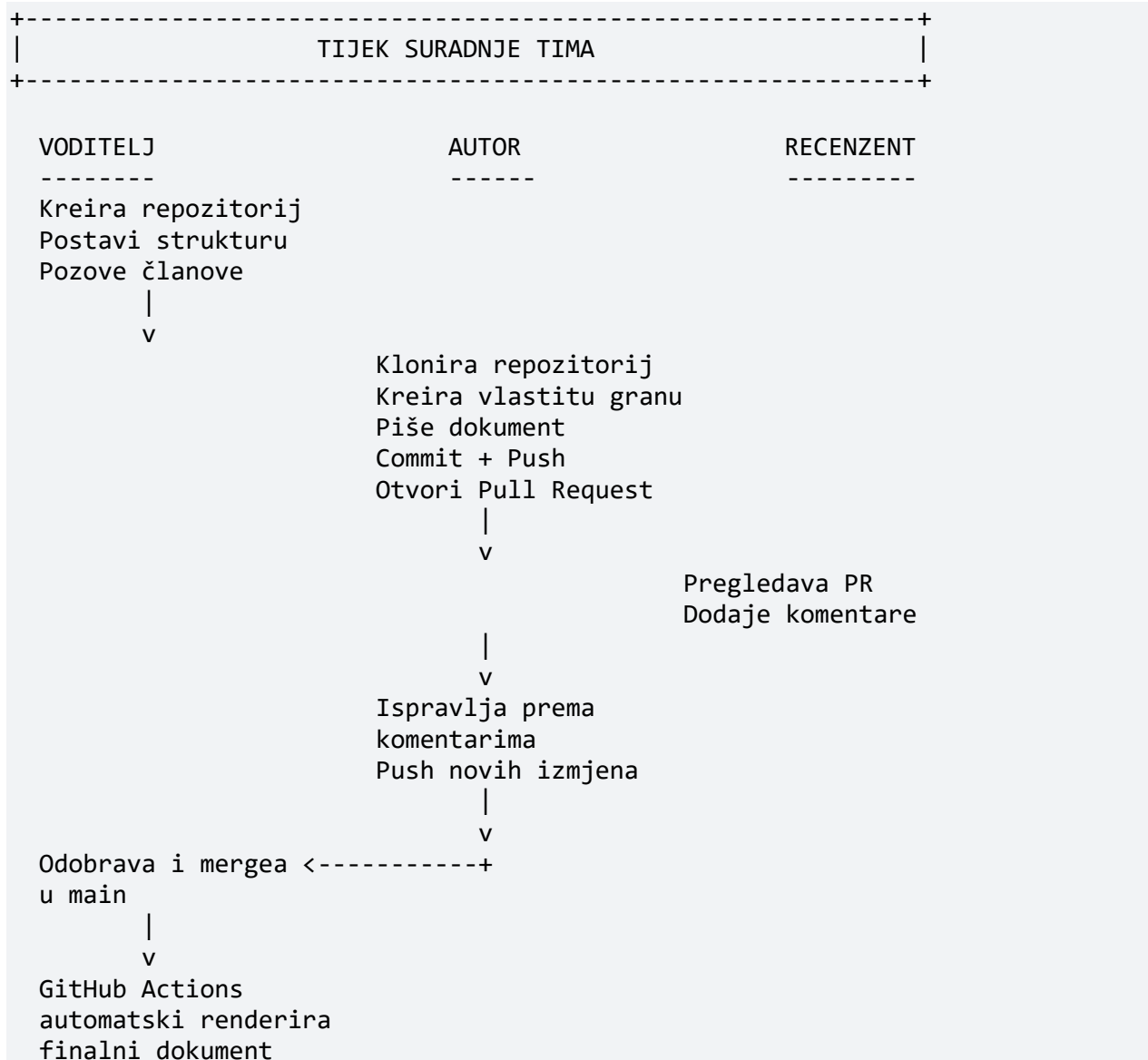
2. Vlasništvo (Jedna osoba = Jedna datoteka)

Budući da je projekt razbijen na module, tim se može dogovoriti tko je “vlasnik” kojeg modula. Ako je Ana zadužena za *Uvod*, nitko drugi u timu **ne smije otvarati niti uređivati** datoteku 01-uvod.qmd dok je Ana na njoj.

Kada odvojimo ljude u zasebne datoteke, Git može savršeno i potpuno automatski spojiti njihov rad bez ikakvih konflikata!

5.3 Praktičan primjer: Tijek suradnje i zaduženja

Ovako izgleda kompletan životni ciklus jednog dokumenta kroz timsku suradnju:



Tablica podjele rada

Prije početka pisanja, projektni menadžer kreira ovakvu tablicu zaduženja (obično u README.md datoteci na GitHubu):

Tablica podjele rada

Član tima	Datoteka	Poglavlje	Status
Ana Kovač	01-uvod.qmd	1. Uvod i kontekst	● U izradi

Član tima	Datoteka	Poglavlje	Status
Pero Novak	02-metodologija.qmd	2. Metodologija provedbe	☐ Nije početo
Marija Horvat	03-rezultati.qmd	3. Rezultati i pokazatelji	☒ Gotovo
Tim	index.md	Naslovnica i meta-podaci	<input type="lock"/> Zaključano

5.4 Oprez: Što je index.md i tko je njegov vlasnik?

U tablici iznad vidiš da je datoteka index.md “zaključana”. Ako ju otvoriš, na samom vrhu vidjet ćeš blok koda oivičen s tri crtice:

```
***
title: "Prijava EU Projekta"
author: "Tim za razvoj"
date: "2023-10-25"
***
```

Ovo se zovu **metapodaci (YAML)**. Quarto će automatski uzeti ove informacije i pozicionirati ih na naslovnicu tvog gotovog Word/PDF dokumenta.

Najčešća početnička greška u Docs-as-Code! U praksi, ovu datoteku uređuje isključivo **Voditelj projekta**. Ostali članovi tima je NE bi trebali dirati! Ako dvije osobe iz tima istovremeno promijene datum u YAML bloku, nastat će Merge Conflict koji će prekinuti automatsko generiranje Word dokumenta za cijeli tim!

5.5 Zlatna pravila za miran san (Bez konflikata!)

Evo konkretnih navika koje konflikte svode na nulu:

Navika	Zašto pomaže
Pull svako jutro (Preuzimanje)	Kad otvoriš Obsidian, prvo napravi <i>Git: Pull</i> . Uvijek moraš imati najnoviju verziju rada tvojih kolega.
Poštuj granice datoteka	Radi samo u datotekama koje su ti dodijeljene. Ne ispravljaj tipfelere u tuđem tekstu dok on radi!
Commit često	Najgora navika je pisanje 4 sata bez ijednog Committa. Što su Commiti manji i češći, konflikti se teže događaju.
Komunikacija spašava	Nemaš strpljenja i moraš hitno nešto promijeniti u tuđoj datoteci? Pošalji poruku na Teams/Slack: “ <i>Hej Pero, planiram dodati odlomak u tvoju 02-metodologija.qmd. Mogu li napraviti push?</i> ”

(Napomena: Konačna i najviša zaštita od konflikata je rad na vlastitoj **grani**, što detaljno učimo u sljedećem poglavlju!)

5.6 Test – Simulacija dodjele vlasništva

Da bismo tehnički potvrdili ovaj koncept, napravimo kratku vježbu.

1. Ti preuzmi odgovornost za poglavlje Uvod.
2. U Obsidianu, klikni na ikonu “Nova bilješka” (New note) ili pritisni Ctrl+N.
3. Preimenuj bilješku u 01-uvod (Obsidian će automatski dodati .md).
4. Upiši naslov (# Uvod) i par rečenica teksta.
5. Napravi Commit i Push.

Sve dok pišeš isključivo unutar svog 01-uvod dokumenta, tehnički si apsolutno siguran/na od konflikata s ostalim članovima tima!

☒ Što smo naučili u ovom poglavlju

Koncept	Objašnjenje
Merge Conflict	Nastaje kada dvije osobe mijenjaju isti redak u istoj datoteci. Rješava se ručno.
Podjela po datotekama	Projekt je “knjiga od Lego kockica” - svaki član zadužen je za određene datoteke.
Vlasništvo nad datotekama	Timski dogovor (1 osoba = 1 datoteka), a ne tehničko ograničenje Git-a.
YAML metapodaci (index.md)	Podaci za naslovnicu knjige. Smije ih mijenjati isključivo voditelj!
Prevenција konflikata	<i>Pull</i> svako jutro + česti, mali <i>Commitovi</i> + poštivanje tuđih datoteka.

<- Prethodno poglavlje: [Poglavljje 4 – Kloniranje repozitorija](#) Sljedeće poglavlje: [Poglavljje 6 – Branching: Tvoj izolirani prostor za rad](#) ->

Poglavljje 6 - Branching: radna grana

 **Procijenjeno vrijeme:** 15–20 minuta  **Preduvjet:** Vault otvoren, Obsidian Git plugin konfiguriran (Poglavljje 3 i 4)

6.1 Što je branch i zašto ga koristimo?

Analogija

Zamisli da radiš na važnom dokumentu i tvoj šef ti kaže: “*Napravi kopiju ovog dokumenta, eksperimentiraj koliko hoćeš — original ostaje netaknut sve dok ne budemo zadovoljni tvojom verzijom.*”

Branch je točno to — tvoja radna kopija cijelog repozitorija u kojoj možeš pisati, mijenjati i brisati što god hoćeš, a da ne utječeš na originalnu verziju ili na rad kolega.

Kada je tvoj rad gotov i pregledan, tek tada se spaja s originalnom verzijom.

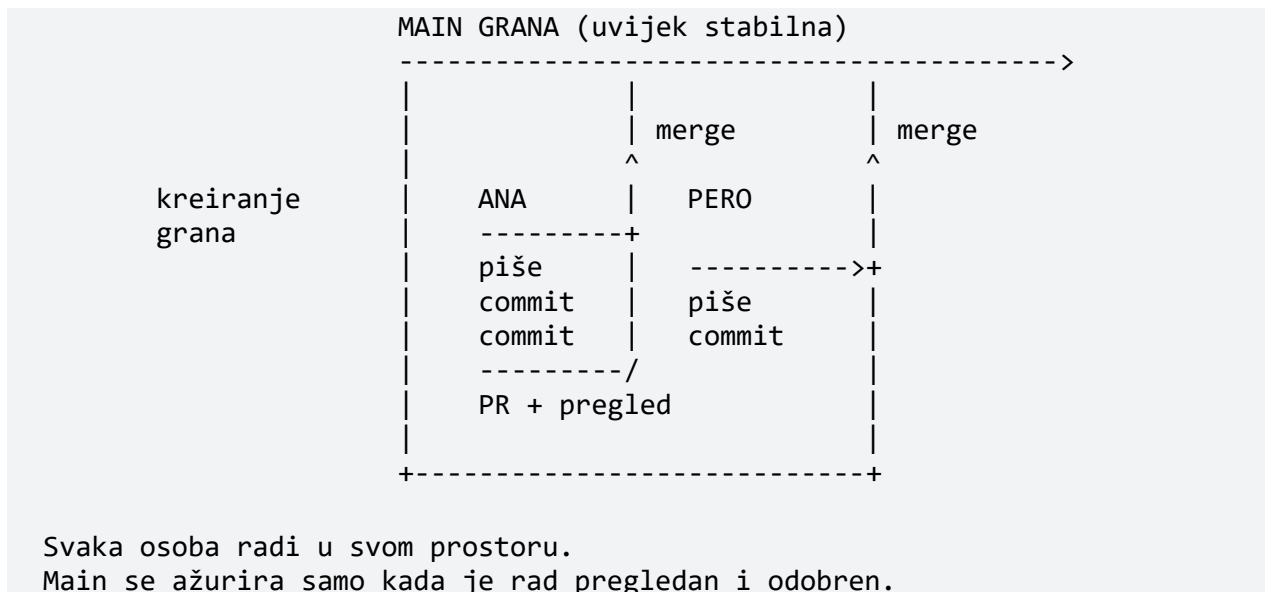
Zašto ne pisati direktno na main?

main je **glavna grana** — ona koja uvijek sadrži zadnju odobrenu, ispravnu verziju dokumentacije. Pravilo je jednostavno:

Na main granu nikad ne pišemo direktno — samo spajamo odobreni rad.

Zašto? Jer ako više ljudi piše direktno na main istovremeno, dolazi do konflikata i nitko ne zna koja je verzija ispravna. Grane su rješenje tog problema.

6.2 Vizualni prikaz grananja



Životni ciklus jednog brancha

1. Kreiraš branch -> "ana-kovac/uvod"
2. Pišeš i commitaš -> branch raste
3. Otvoriš PR -> kolega pregledava

- | | |
|--------------------|-----------------------------------|
| 4. Merge u main | -> tvoj rad je dio glavne verzije |
| 5. Branch se briše | -> posao završen |

6.3 Pravila imenovanja brancheva

Dobro ime brancha odmah govori **tko** radi **što**. Koristimo ovaj format:

ime-prezime/naziv-zadatka

Primjeri

Dobro	Loše
ana-kovac/uvod	moja-grana
pero-novak/metodologija	izmjene
mj-horvat/rezultati-q3	branch1
ana-kovac/ispravak-tablice	novo

Pravila formata

- Samo **mala slova**
- Koristiti **crtice** umjesto razmaka ili podcrta
- **Bez naglašenih slova** (č, ć, š, ž, đ) — mogu uzrokovati probleme
- **Kratko i opisno** — maksimalno 3–4 riječi nakon kose crte

6.4 Kreiranje brancha u Obsidianu

Obsidian Git ne prikazuje gumbe za upravljanje granama u Source Control panelu — za to koristimo **Command Palette**, ugrađenu Obsidian funkcionalnost za pokretanje naredbi.

Korak 1: Otvori Command Palette

Pritisni **Ctrl + P**

Otvara se traka za pretraživanje naredbi na vrhu ekrana.

Korak 2: Kreiraj novi branch

1. Upiši: `git branch`
2. U rezultatima odaberi: **“Obsidian Git: Create new branch”**
3. Pritisni **Enter**

Korak 3: Upiši naziv brancha

Pojavljuje se polje za unos naziva:

```
+-----+
| Enter name of new branch |
| > ana-kovac/uvod        |
+-----+
```

Upiši naziv prema pravilima iz poglavlja 6.3 i pritisni **Enter**.

✅ **Provjera uspjeha:** U statusnoj traci na dnu Obsidiana naziv brancha se promijenio:

```
Prije: Current branch: main | v0 ^0
Nakon: Current branch: ana-kovac/uvod | v0 ^0
```

💡 **Savjet:** Uvijek provjeri statusnu traku prije nego počneš pisati — mora pisati tvoj branch, ne main.

6.5 Prebacivanje između postojećih brancheva

Ako trebaš privremeno pogledati tuđi branch ili se prebaciti na drugi vlastiti:

1. Pritisni **Ctrl + P**
2. Upiši: `git checkout`
3. Odaberi: **“Obsidian Git: Switch branch”**
4. Iz popisa odaberi branch na koji se želiš prebaciti

⚠️ **Upozorenje:** Prije prebacivanja brancha uvijek napravi **commit** svih svojih promjena. Ako imaš nesačuvane izmjene i prebacuješ branch, Obsidian Git može javiti grešku ili — u rijetkim slučajevima — promjene mogu biti vidljive na krivom branchu.

6.6 Povratak na main

Kada završiš s radom na svom branchu i merge je obavljen (Poglavlje 9), možeš se vratiti na main:

1. **Ctrl + P** -> `git checkout` -> **“Obsidian Git: Switch branch”**
2. Iz popisa odaberi **main**
3. Odmah napravi **Pull** da preuzmeš zadnje odobrene promjene

6.7 Git CLI alternativa

Ako iz bilo kojeg razloga Command Palette ne funkcioniра, iste operacije možeš napraviti kroz Command Prompt.

Otvori Command Prompt u folderu repozitorija:

```
cd C:\Projekti\EU-Project-Template
```

Kreiranje i prebacivanje na novi branch

```
git checkout -b ana-kovac/uvod
```

i Zastavica -b znači “kreiraj novi branch i odmah se prebaci na njega” — sve u jednoj naredbi.

Pregled svih brancheva

```
git branch
```

Rezultat:

```
main
* ana-kovac/uvod      <- zvjezdica označava na kojem si trenutno
pero-novak/metodologija
```

Prebacivanje na postojeći branch

```
git checkout main
```

Slanje novog brancha na GitHub

Kada prvi put pushaš novi branch na GitHub, treba mu reći gdje:

```
git push -u origin ana-kovac/uvod
```

i Zastavica -u origin potrebna je samo **prvi put** za novi branch. Sve sljedeće pusheve s tog brancha radiš jednostavno s `git push`.

Nakon prvog pusha, Obsidian Git plugin prepoznaje branch i možeš nastaviti koristiti gumbe u sučelju kao i obično.

6.8 Pregled: naredbe na jednom mjestu

Radnja	Obsidian (Ctrl+P)	Git CLI
Kreiraj novi branch	Obsidian Git: Create new branch	<code>git checkout -b ime/naziv</code>
Prebaci se na branch	Obsidian Git: Switch branch	<code>git checkout ime-brancha</code>
Pregled svih brancheva	—	<code>git branch</code>
Provjeri trenutni branch	Statusna traka (dno ekrana)	<code>git branch</code> (zvjezdica)
Pošalji branch na GitHub	Push gumb (Source Control)	<code>git push -u origin ime-brancha</code>

☑ Što smo naučili u ovom poglavlju

Koncept	Objašnjenje
Branch	Izolirana radna kopija repozitorija za jednog autora
main	Glavna grana — uvijek stabilna, nikad se ne piše direktno
Imenovanje	Format ime-prezime/naziv-zadatka, bez naglašenih slova
Command Palette	Ctrl+P — centralno mjesto za sve Git operacije u Obsidianu
Statusna traka	Uvijek provjeri naziv brancha prije pisanja
Prvi push	Novi branch treba <code>git push -u origin naziv</code> prvi put

Zlatno pravilo brancheva: Jedan zadatak = jedan branch. Kada je zadatak gotov i mergean, branch se briše i kreiraš novi za sljedeći zadatak. Ne nakupljaj stare brancheve — postaje zbunjujuće.

<- **Prethodno poglavlje:** [Poglavlje 5 – Podjela rada](#) **Sljedeće poglavlje:** [Poglavlje 7 – Pisanje dokumenta u Obsidianu](#) ->

Poglavlje 7 - Pisanje dokumenta

🕒 **Procijenjeno vrijeme:** 20–30 minuta 📄 **Preduvjet:** Vault otvoren, vlastiti branch kreiran (Poglavlje 6)

7.1 Markdown – format u kojemu pišemo

Što je Markdown?

Markdown je način pisanja teksta gdje **formatiranje opisuješ jednostavnim znakovima** unutar samog teksta — bez klikanja na gumbe kao u Wordu.

Zašto? Jer je tekst bez skrivenih kodova format koji Git može pratiti, uspoređivati i spajati. Word .docx datoteke su iznutra kompleksni XML — Git ih ne može čitati smisleno.

Kako to izgleda u praksi

Pišeš ovo:

Uvod projekta

Projekt se provodi u **tri faze** tijekom **2024. godine**.

Ciljevi su:

- Smanjenje potrošnje energije za 30%
- Ugradnja obnovljivih izvora
- Edukacija korisnika

Obsidian ti prikazuje ovo (Reading View):

Uvod projekta

Projekt se provodi u **tri faze** tijekom **2024. godine**.

Ciljevi su: - Smanjenje potrošnje energije za 30% - Ugradnja obnovljivih izvora - Edukacija korisnika

A Quarto od toga generira lijepo formatirani Word dokument s automatski primijenjenim stilovima.

7.2 Markdown osnove – sve što trebaš znati

Naslovi

```
# Naslov razine 1 (Heading 1 u Wordu)
## Naslov razine 2 (Heading 2 u Wordu)
### Naslov razine 3 (Heading 3 u Wordu)
```

⚠ **Važno:** Između # i teksta naslova mora biti **razmak**. #Naslov ne funkcionira, #Naslov funkcionira.

Isticanje teksta

```
**podebljani tekst**
*kurziv*
***podebljani kurziv***
```

Prikazuje se kao: **podebljani tekst**, *kurziv*, ***podebljani kurziv***

Nabrajanje – liste

Nenumerirana lista (bullet points):

- Prva stavka
- Druga stavka
 - Podstavka (uvučena s 2 razmaka)
 - Još jedna podstavka
- Treća stavka

Numerirana lista:

1. Prva stavka
2. Druga stavka
3. Treća stavka

💡 **Savjet:** Kod numerirane liste ne moraš paziti na redoslijed brojeva — možeš pisati sve 1. i Quarto će automatski ispravno numerirati pri renderiranju.

Tablice

Stupac 1	Stupac 2	Stupac 3
Sadržaj ćelije	Sadržaj ćelije	Sadržaj ćelije
Drugi redak	Drugi redak	Drugi redak

Prikazuje se kao:

Stupac 1	Stupac 2	Stupac 3
Sadržaj ćelije	Sadržaj ćelije	Sadržaj ćelije
Drugi redak	Drugi redak	Drugi redak

💡 **Savjet:** Ne moraš ručno poravnavati crtice i stupce — Obsidian to radi automatski kada pritisneš Tab unutar tablice. Možeš i koristiti plugin “**Advanced Tables**” koji olakšava rad s tablicama.

Linkovi i slike

Link:

[tekst koji se prikazuje](https://www.primjer.com)

Slika:

![opis slike](putanja/do/slike.png)

Savjet za slike: Slike povuci i ispusti (drag & drop) direktno u Obsidian editor. Plugin će automatski kopirati datoteku na ispravnu lokaciju i umetnuti Markdown kod.

Blokovi koda i napomene

Inline kod (za kratke naredbe unutar teksta):

```
Pokreni naredbu `quarto render` u terminalu.
```

Blok koda (za dulje isječke):

```
```\ngit commit -m "opis promjene"\ngit push origin naziv-brancha\n```
```

**Horizontalna crta** (za vizualno odvajanje sekcija):

```

```

---

## Quarto specifične napomene (callouts)

Quarto dodaje posebne blokove koji se lijepo renderiraju u finalnom dokumentu:

```
::: {.callout-note}\nOvo je informativna napomena.\n:::\n\n::: {.callout-warning}\nOvo je upozorenje.\n:::\n\n::: {.callout-important}\nOvo je važna informacija.\n:::
```

---

## Matematičke formule i izrazi

Quarto podržava pisanje matematičkih formula u **LaTeX notaciji**. Ovo je korisno za EU projektne dokumente koji uključuju statističke pokazatelje, financijske formule ili metodološke izračune.

*Inline formula (unutar teksta)*

Za formulu unutar rečenice koristi **jednu dolarsku oznaku** ( $...$ ):

Stopa apsorpcije izračunava se kao  $A = \frac{I}{P} \times 100\%$ , gdje je  $I$  iznos isplaćenih sredstava, a  $P$  ukupni proračun projekta.

Prikazuje se kao:

Stopa apsorpcije izračunava se kao  $A = \frac{I}{P} \times 100\%$ , gdje je  $I$  iznos isplaćenih sredstava, a  $P$  ukupni proračun projekta.

---

#### Izdvojena formula (u zasebnom retku)

Za formulu koja stoji samostalno, na sredini stranice, koristi **dvostruku dolarsku oznaku** ( $\$ \dots \$$ ):

```

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

```

Prikazuje se kao:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

---

#### Primjeri iz EU projektne dokumentacije

##### Primjeri matematičkih formula za EU projekte

Što izračunavaš	Markdown kod
Postotni udio	$p = \frac{a}{b} \times 100\%$
Prosjek	$\bar{x} = \frac{\sum x_i}{n}$
Standardna devijacija	$\sigma = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n}}$
Složena kamatna stopa	$A = P(1 + r)^t$
Energetska učinkovitost	$\eta = \frac{W_{\text{korisni}}}{W_{\text{ukupni}}} \times 100\%$

---

#### Česti LaTeX simboli

##### Česti LaTeX simboli

Simbol	Kod	Primjer
Razlomak	$\frac{a}{b}$	$\frac{a}{b}$



Simbol	Kod	Primjer
Suma	<code>\sum_{i=1}^n</code>	$\sum_{i=1}^n$
Kvadratni korijen	<code>\sqrt{x}</code>	$\sqrt{x}$
Eksponent	<code>x^{2}</code>	$x^2$
Indeks	<code>x_{i}</code>	$x_i$
Postotak	<code>\%</code>	$\%$
Srednja vrijednost	<code>\bar{x}</code>	$\bar{x}$
Delta (promjena)	<code>\Delta</code>	$\Delta$
Veće/manje	<code>\geq, \leq</code>	$\geq, \leq$
Priblizno	<code>\approx</code>	$\approx$

 **Savjet:** Obsidian u Reading View prikazuje formule renderiranjem u stvarnom vremenu — vidiš rezultat dok pišeš. Ako formula ne izgleda ispravno, najčešći uzrok je pogrešna sintaksa u LaTeX kodu (nedostaje `}` ili `{}`).

**Napomena za Word output:** Quarto pri renderiranju u `.docx` pretvara LaTeX formule u Word-ove native matematičke objekte (OMath). Ne moraš ništa posebno raditi — formula u Markdownu automatski postaje ispravna formula u Wordu.

## 7.3 Struktura `.qmd` datoteke

### YAML zaglavlje (metadata)

Na samom vrhu svake `.qmd` datoteke nalazi se blok između trojnih crtica:

```

title: "Naziv poglavlja"
author: "Ime Prezime"
date: "2024-03-15"

```

 **Upozorenje:** YAML zaglavlje mora biti **točno na prvom retku** datoteke, bez ikakvog teksta ispred. Ako pomiješaš format (npr. zaboraviš zatvoriti `---`), Quarto neće moći renderirati dokument.

### Kompletna struktura datoteke

```

title: "1. Uvod i kontekst projekta"
author: "Ana Kovač"
date: "2024-03-15"

```

## ## 1.1 Kontekst projekta

Ovdje počinješ pisati sadržaj...

## ## 1.2 Ciljevi

- Cilj 1
- Cilj 2

## ## 1.3 Projektni tim

Uloga	Ime	Organizacija
-----	-----	-----
Voditelj	Ana Kovač	FGAG

---

## 7.4 Korištenje predloška iz repozitorija

Repozitorij sadrži predloške u folderu templates/ koji definiraju strukturu dokumenta.

### Kako koristiti predložak

1. U Obsidianu desni klik na folder docs/
2. Odaberi **“New note”**
3. Daj datoteci naziv prema konvenciji projekta (npr. 01-uvod.qmd)
4. Na vrh datoteke kopiraj YAML zaglavlje iz predloška
5. Počni pisati sadržaj ispod zaglavlja

💡 **Savjet:** Otvori postojeću .qmd datoteku iz repozitorija kao referencu — vidiš točan format zaglavlja i strukturu koji projekt koristi.

---

## 7.5 Dva načina prikaza u Obsidianu

Obsidian nudi dva načina rada s datotekom:

### Editing View (uređivanje)

Vidiš sirovi Markdown tekst s oznakama (\*\*, #, -). U ovom modu pišeš.

### Reading View (čitanje)

Vidiš formatiran tekst — naslovi su naslovi, bold je bold, tablice su tablice. Korisno za provjeru izgleda.

**Prebacivanje između modova:** - Klikni na ikonu **knjige** u gornjem desnom kutu editora - Ili pritisni Ctrl + E

💡 **Savjet:** Postoji i **Live Preview** mod koji prikazuje formatiran tekst dok pišeš — kao Word. Uključiš ga u Settings -> Editor -> “Default editing mode” -> “Live Preview”. Preporučujemo za početnike.

---

## 7.6 Preporučeni tijek pisanja

Svaki radni dan (ili radna sesija) trebao bi izgledati ovako:

+-----+   DNEVNI TIJEK PISANJA   +-----+	
POCETAK RADA ----- 1. Otvori Obsidian 2. Provjeri statusnu traku -> jesi li na svom branchu? 3. Pull (v gumb) -> preuzmi izmjene kolega 4. Otvori svoju .qmd datoteku i počni pisati  TIJEKOM PISANJA (svakih 30-60 minuta ili po cjelini) ----- 5. Stage All -> Commit -> poruka: sto si napisao/la? (npr. "Dodao uvod 1.2")  KRAJ RADNOG DANA ----- 6. Zadnji Commit 7. Push (^ gumb) -> pošalji sve na GitHub 8. Zatvori Obsidian	

💡 **Zašto commit svakih 30–60 minuta?** Svaki commit je točka na koju se možeš **vratiti** ako nešto krene naopako. Ako pišeš 3 sata bez commita i slučajno obrišeš poglavlje, nemaš se gdje vratiti.

---

## 7.7 Commit i push kroz Obsidian plugin

### Korak po korak

#### 1. Stage All — označi promjene za commit

U Source Control panelu (Ctrl+Shift+G) klikni “**Stage All**”.

Sve datoteke s oznakom [M] (Modified) ili [U] (Untracked) prelaze u sekciju “Staged Changes”.

**i Što znači “stage”?** Git razlikuje “što si promijenio” od “što šalješ u ovaj commit”. Stage je kao stavljanje stvari u kovčeg — odabiraš što ide zajedno.

## 2. Commit — spremi s opisom

Klikni “Commit”.

Plugin automatski generira poruku s datumom i brojem datoteka (prema formatu iz Poglavlja 3.5). Ako hoćeš vlastitu poruku, klikni na polje iznad gumba Commit i upiši ju.

### Savjeti za dobre commit poruke

Commit poruka treba odgovoriti na: “Što sam napravio/la?”

**Zlatno pravilo:** Zamisli da rečenica počinje s “Ako primijenim ovaj commit, on će...”. Tvoja poruka treba biti logičan nastavak te rečenice.

Dobra poruka (Preporuka)	Loša poruka (Ne raditi)
Dodati poglavlje 1.2 - Ciljevi projekta	izmjene
Ispraviti tablicu projektnog tima	save
Obrisati stare napomene prema komentarima Marka	aaa
Završiti poglavlje Metodologija	update

## 3. Push — pošalji na GitHub

Ako je “Push on commit” uključen (Poglavlje 3.5), push se dogodi automatski. Ako nije, klikni gumb **Push** u Source Control panelu.

**✓ Provjera uspjeha:** U statusnoj traci `^0` znači da nema čekajućih commitova za push — sve je na GitHubu.

---

## 7.8 Git CLI alternativa

Ako preferiraš upisivati naredbe u Command Prompt:

```
1. Dodaj sve izmijenjene datoteke
git add .

2. Napravi commit s porukom
git commit -m "Dodati poglavlje 1.2 - Ciljevi projekta"

3. Pošalji na GitHub
git push origin ana-kovac/uvod
```

💡 **Savjet:** `git add .` (s točkom na kraju) dodaje **sve** izmijenjene datoteke odjednom. Točka znači “sve u trenutnom folderu i podfolderima”.

---

## ☑ Što smo naučili u ovom poglavlju

Koncept	Objašnjenje
<b>Markdown</b>	Jednostavan tekstualni format za pisanje s oznakama za formatiranje
<b>YAML zaglavlje</b>	Metadata blok na vrhu svake <code>.qmd</code> datoteke
<b>.qmd datoteka</b>	Quarto Markdown datoteka — naša osnovna jedinica dokumenta
<b>Stage</b>	Odabir datoteka koje idu u sljedeći commit
<b>Commit</b>	Snimanje trenutnog stanja s opisnom porukom
<b>Push</b>	Slanje commitova na GitHub
<b>Dnevni ritam</b>	Pull -> piši -> commit (često) -> push (kraj dana)

---

<- **Prethodno poglavlje:** [Poglavljje 6 – Branching](#) **Sljedeće poglavlje:** [Poglavljje 8 – Pull Request: predaja rada na pregled](#) ->

## Poglavljje 8 - Pull Request: predaja rada na pregled

🕒 **Procijenjeno vrijeme:** 15–20 minuta 📋 **Preduvjet:** Završeno pisanje na vlastitom branchu, barem jedan commit pushean na GitHub (Poglavljje 7)

---

### 8.1 Što je Pull Request i zašto je važan?

#### Definicija

**Pull Request** (skraćeno: PR) je formalni zahtjev da se tvoje promjene s radnog brancha **spoje u main granu**.

Naziv može zbuniti — ne radi se o tome da ti nešto “povlačiš” (pull), nego da **tražiš od tima da povuče tvoje promjene** u glavnu verziju.

#### Zašto ne spajamo direktno?

Jer svaka promjena koja ulazi u `main` mora biti:

- Pregledana od bar jedne druge osobe
- Bez tehničkih grešaka (ispravan Markdown, YAML zaglavlje)

- Sadržajno odobrena (točnost, stil, konzistentnost s ostatkom dokumenta)

PR je **kapija kontrole kvalitete** — ništa ne ulazi u službenu verziju bez pregleda.

## Analogija

Zamisli PR kao **predaju zadaće profesoru**: - Ti predaješ (otvaraš PR) - Profesor pregledava i komentira (Review) - Ti ispravljaš prema komentarima (novi commits na isti branch) - Profesor odobrava i “zaključuje” (Merge)

Razlika od klasičnog pregleda dokumenta: sve je vidljivo, sve je zabilježeno, nema e-mail pingponga.

---

## 8.2 Uloge u procesu pregleda

Uloga	Tko	Što radi
<b>Autor</b>	Onaj tko je pisao dokument	Otvora PR, odgovara na komentare, ispravlja
<b>Recenzent</b>	Kolega iz tima ili voditelj	Pregledava, komentira, odobrava ili traži izmjene
<b>Maintainer</b>	Voditelj projekta	Ima pravo mergati u main — može biti ista osoba kao recenzent

💡 **Savjet:** Dogovorite u timu tko je maintainer — ta osoba ima finalnu odgovornost za kvalitetu main grane. Obično je to voditelj dokumentacije.

---

## 8.3 Kada otvoriti Pull Request?

PR otvori kada: - Završiš poglavlje ili logičnu cjelinu (ne čekaj da bude “savršeno”) - Sve promjene su pushane na GitHub - Lokalno si provjerio/la da Markdown izgleda ispravno (Reading View u Obsidianu)

Ne čekaj da bude sto posto gotovo — PR je i alat za **rano dobivanje povratnih informacija**. Bolje kratki PR s jednim poglavljem nego jedan veliki PR s cijelim dokumentom.

---

## 8.4 Otvaranje Pull Requesta na GitHubu

### Korak 1: Idi na GitHub repozitorij

1. Otvori preglednik i idi na: <https://github.com/FGAG-docs/EU-Project-Template>
2. Prijavi se ako nisi prijavljen/a

## Korak 2: GitHub primijeti tvoj branch

Kada pushaš branch na GitHub, stranica repozitorija automatski prikazuje žuti banner:

```
+-----+
| [!] ana-kovac/uvod had recent pushes 2 minutes ago |
| [Compare & pull request] |
+-----+
```

Klikni na **“Compare & pull request”**.

**Ako banneru nema:** Klikni na karticu **“Pull requests”** -> zeleni gumb **“New pull request”** -> u padajućem izborniku **“compare”** odaberi svoj branch -> klikni **“Create pull request”**.

## Korak 3: Ispuni obrazac Pull Requesta

Otvora se obrazac s nekoliko polja:

---

### Title (Naslov)

Naslov PR-a treba jasno opisati što si napravio/la:

Dobro	Loše
Poglavlje 1 - Uvod i kontekst projekta	moje izmjene
Ispravak tablice projektnog tima (Pogl. 2)	update
Dodano poglavlje Metodologija - prva verzija	PR

---

### Description (Opis)

Opis je prostor gdje objašnjavaš recenzentu što treba pregledati. Preporučeni format:

```
Što je napravljeno
- Napisano poglavlje 1.1 Kontekst projekta
- Napisano poglavlje 1.2 Ciljevi
- Dodana tablica projektnog tima

Što treba pregledati
- Jesu li ciljevi usklađeni s projektnim prijedlogom?
- Provjeri tablicu - nisam siguran/na za nazive organizacija

Što još nije gotovo (ako je primjenjivo)
- Poglavlje 1.3 (Dionici) - dolazi u sljedećem PR-u
```

---

### Reviewers (Recenzenti)

Na desnoj strani obrasca, pod “Reviewers”: 1. Klikni na zupčanik  pored “Reviewers” 2. Upiši GitHub korisničko ime kolege koji treba pregledati 3. Odaberi ga s popisa

 **Savjet:** Recenzent dobiva e-mail obavijest. Ako žuriš, dodatno ga obavijesti porukama — GitHub notifikacije ponekad završe u spamu.

---

### Labels (Oznake) — opcionalno

Labels pomažu organizirati PR-ove u većim projektima. Uobičajene oznake: - in-review — čeka pregled - needs-revision — treba ispravke - ready-to-merge — odobreno

Ako tim nije postavio labels, preskoči ovaj korak.

---

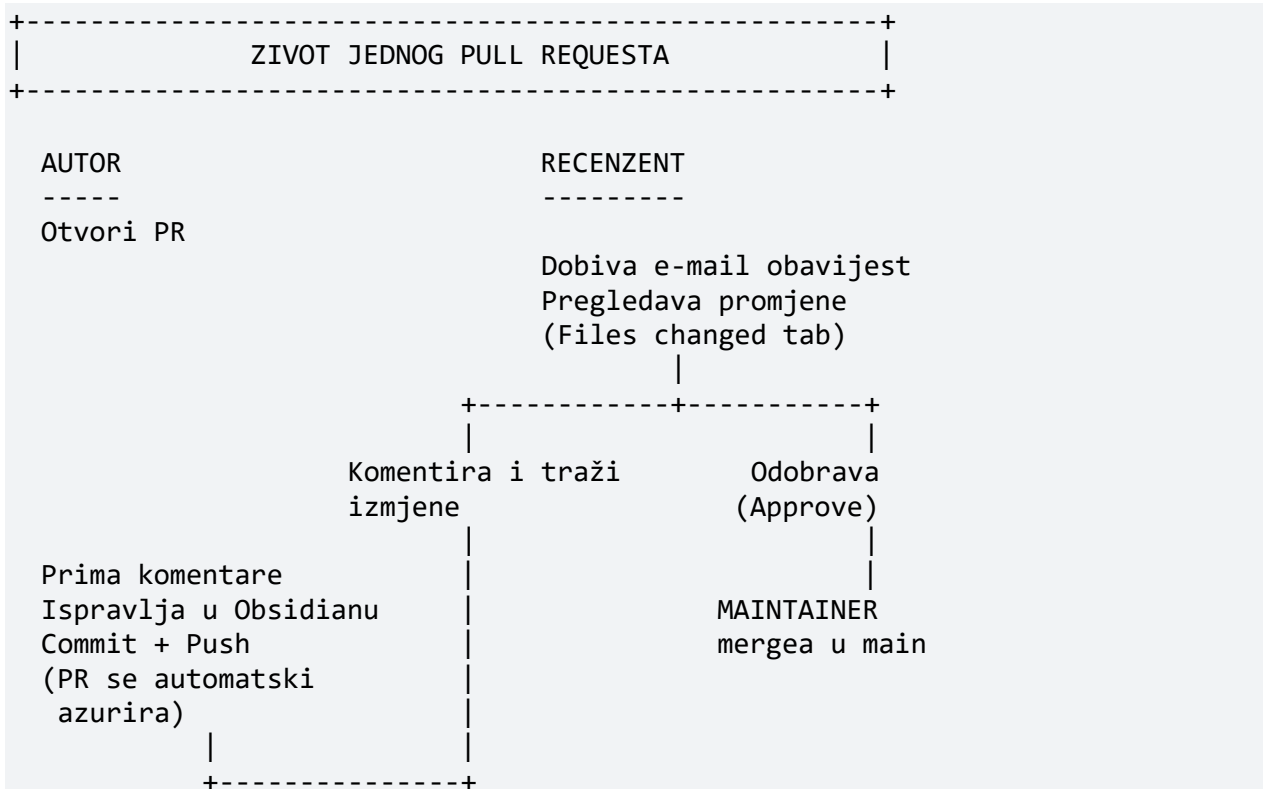
## Korak 4: Pošalji Pull Request

Klikni zeleni gumb “**Create pull request**”.

 **Provjera uspjeha:** PR je otvoren i vidljiv pod karticom “Pull requests” repozitorija. Recenzent je dobio obavijest.

---

## 8.5 Što se događa nakon otvaranja PR-a?





(ciklus se ponavlja  
dok nije odobreno)

---

## 8.6 Kako reagirati na komentare recenzenta

### Gdje vidiš komentare

1. Na GitHubu, otvori PR
2. Klikni na karticu **"Files changed"** — vidiš točno koje linije su komentirane
3. Klikni na karticu **"Conversation"** — vidiš sve komentare kronološki

### Kako ispraviti prema komentarima

1. Vрати se u **Obsidian**
2. Otvori datoteku koja je komentirana
3. Napravi tražene izmjene
4. **Stage All -> Commit -> Push**

**i Automatsko ažuriranje PR-a:** Ne moraš otvarati novi PR. Svaki novi commit na isti branch automatski se pojavljuje u otvorenom PR-u. Recenzent vidi sve promjene u realnom vremenu.

5. Na GitHubu, odgovori na komentar: klikni **"Resolve conversation"** kada si riješio/la tu točku

### Bonton PR komunikacije

Situacija	Preporučeni odgovor
Slažeš se s komentarom	Napravi izmjenu, klikni "Resolve conversation"
Ne razumiješ komentar	Odgovori pitanjem u komentaru, ne pretpostavljaj
Ne slažeš se	Objasni razlog u komentaru — odluku donosi maintainer
Komentar je zastario	Napiši "Riješeno u commitu abc123" i zatvori

---

## 8.7 Provjera prije nego pushaš PR na recenziju

Kratka kontrolna lista koju prođeš sam/a prije nego tražiš pregled:

```
[] Sve datoteke su commitane i pushane
[] Naslov PR-a je jasan i opisuje sadržaj
[] Opis objašnjava što treba pregledati
[] Odabran je recenzent
[] Otvorio/la sam "Files changed" i pregledao/la vlastite promjene
[] Nema slučajno dodanih testnih redaka ili komentara tipa "TODO"
[] YAML zaglavlje je ispravno (naslov, autor, datum)
```

---

## ☑ Što smo naučili u ovom poglavlju

Koncept	Objašnjenje
<b>Pull Request (PR)</b>	Formalni zahtjev za spajanje brancha u main, s procesom pregleda
<b>Reviewer</b>	Osoba koja pregledava i komentira PR
<b>Maintainer</b>	Osoba s pravom merjanja u main
<b>Files changed</b>	GitHub prikaz točno što se promijenilo, redak po redak
<b>Resolve conversation</b>	Označavanje komentara kao riješenog
<b>PR se ažurira automatski</b>	Svaki novi commit na branch pojavljuje se u otvorenom PR-u

<- Prethodno poglavlje: [Poglavlje 7 – Pisanje dokumenta](#) Sljedeće poglavlje: [Poglavlje 9 – Merge u main: spajanje promjena](#) ->

## Poglavlje 9 - Merge u main: spajanje promjena

🕒 **Procijenjeno vrijeme:** 15–20 minuta 📄 **Preduvjet:** Otvoren i pregledan Pull Request (Poglavlje 8)

### 9.1 Što znači merge i kada se radi?

**Merge** (hrv. *spajanje*) je postupak kojim se promjene s radnog brancha trajno ugrađuju u main granu.

Nakon mergea: - Sve što si napisao/la postaje dio **službene verzije** dokumenta - main grana sadrži tvoj doprinos zajedno s doprinosima svih ostalih - GitHub Actions automatski pokreće Quarto render i generira novi finalni dokument

#### Kada je pravo vrijeme za merge?

Merge se radi **isključivo** kada su ispunjena sva tri uvjeta:

- [ ] Recenzent je pregledao PR i kliknuo "Approve"
- [ ] Svi komentari su označeni kao riješeni (Resolve conversation)
- [ ] Nema konflikata s main granom (GitHub će jasno javiti ako ih ima)

⚠ **Upozorenje:** Nikad ne mergeaj vlastiti PR bez pregleda drugog člana tima — čak i ako si siguran/na da je sve ispravno. Pregled nije samo o greškama, nego i o konzistentnosti cijelog dokumenta.

## 9.2 Tko ima pravo mergeati?

Pravo merjanja ima **maintainer** — osoba s administratorskim ili write pristupom repozitoriju.

U praksi, to znači:

Veličina tima	Tko mergea
Mali tim (2–3 osobe)	Svaki član može mergeati tuđe PR-ove
Srednji tim (4+)	Voditelj dokumentacije mergea sve PR-ove
Formalni projekt	Samo maintainer, uz obavezni Approve

💡 **Preporuka:** Postavi pravilo u timu da **autor ne mergea vlastiti PR**. Četiri oka uvijek vide više od dva.

---

## 9.3 Merge kroz GitHub sučelje

Ovo je najčešći i najjednostavniji način — sve se radi kroz preglednik.

### Korak 1: Otvori Pull Request

1. Idi na <https://github.com/FGAG-docs/EU-Project-Template>
2. Klikni na karticu “**Pull requests**”
3. Odaberi PR koji je spreman za merge

### Korak 2: Provjeri status

Na dnu PR stranice vidiš status provjera:

```
+-----+
| [V] 1 approving review |
| [V] All conversations resolved |
| [V] No merge conflicts |
| [Merge pull request v] |
+-----+
```

Ako su sve tri zelene kvačice — spreman si za merge.

⚠️ **Ako vidiš crveni X uz “Merge conflicts”:** Vidi poglavlje 9.5 — konflikte treba riješiti prije mergea.

### Korak 3: Odaberi tip mergea

Klikni na **strelicu v** pored gumba “Merge pull request” — pojavljuju se tri opcije:

Opcija	Što radi	Kada koristiti
--------	----------	----------------

Opcija	Što radi	Kada koristiti
<b>Create a merge commit</b>	Spaja sve commitove, dodaje jedan “merge commit”	Preporuka za početnike — najjednostavnije
<b>Squash and merge</b>	Sve commitove spljoštava u jedan	Kada imaš puno sitnih “work in progress” commitova koje ne trebaš čuvati
<b>Rebase and merge</b>	Premješta commitove na vrh main grane	Napredno — izbjegavaj dok ne upoznaš Git bolje

**Za ovaj projekt koristite: “Create a merge commit”** — uvijek jasno što se i kada spojilo.

#### Korak 4: Potvrdi merge

1. Klikni **“Merge pull request”**
2. Pojavljuje se polje za potvrdu s automatski generiranom porukom — možeš je ostaviti kakva jest ili kratko urediti
3. Klikni **“Confirm merge”**

✅ **Provjera uspjeha:** GitHub prikazuje:

Pull request successfully merged and closed

#### Korak 5: Obriši branch

Odmah nakon mergea GitHub nudi gumb:

[ Delete branch ]

**Klikni ga.** Radni branch više nije potreban — promjene su u main. Brisanje brancha ne briše promjene, samo čisti repozitorij od zastarjelih grana.

💡 **Zašto brisati branch?** Repozitorij s desetak starih, zastarjelih brancheva postaje zbunjujuć. Čist repozitorij = samo aktivni branchevi su vidljivi.

## 9.4 Ažuriranje lokalnog repozitorija nakon mergea

Nakon što je merge napravljen na GitHubu, tvoje lokalno računalo još uvijek ne zna za tu promjenu. Trebaš preuzeti ažurirani main.

### U Obsidianu

1. Otvori **Command Palette** (Ctrl+P)
2. Upiši: `git checkout`
3. Odaberi: **“Obsidian Git: Switch branch”** -> odaberi main
4. Klikni gumb **v Pull** u Source Control panelu

Lokalni main sada sadrži sve mergane promjene.

## Git CLI alternativa

```
Prebaci se na main granu
git checkout main

Preuzmi najnovije promjene s GitHuba
git pull origin main
```

---

## 9.5 Rješavanje merge konflikata

### Što je konflikt?

Konflikt nastaje kada su **isti redci iste datoteke** izmijenjeni na dva različita brancha koji se pokušavaju spojiti. Git ne zna koju verziju zadržati — traži od tebe da odlučiš.

```
<<<<<<< HEAD (main grana)
Projekt se provodi u tri faze.
=====
Projekt se provodi u četiri faze, od kojih je prva pripremna.
>>>>>>> ana-kovac/uvod (tvoj branch)
```

Git je označio: - Između <<<<<<< HEAD i ===== -> verzija koja je u main - Između ===== i >>>>>>> -> tvoja verzija s brancha

### Kako riješiti konflikt — korak po korak

Konflikte je **najlakše riješiti na GitHubu**, direktno u pregledniku, za jednostavne slučajeve.

#### Na GitHubu:

1. GitHub prikazuje upozorenje na PR stranici:

```
This branch has conflicts that must be resolved
[Resolve conflicts]
```

2. Klikni **“Resolve conflicts”**
3. GitHub prikazuje datoteku s označenim konfliktima
4. Uredi tekst:
  - Obriši linije s <<<<<<<, ===== i >>>>>>>
  - Zadrži onu verziju teksta koja je ispravna (ili kombiniraj obje)
5. Klikni **“Mark as resolved”**
6. Klikni **“Commit merge”**

## 7. Nastavi s normalnim mergeom

⚠ **Za složenije konflikte** koji zahvaćaju više datoteka ili veće dijelove teksta — obavijesti maintainera i riješite zajedno. Ne pogađaj.

### Kako spriječiti konflikte

Konflikti su rijetki ako tim poštuje dogovor o podjeli rada (Poglavlje 5). Ako se ipak pojave češće, najčešći uzrok je:

- Dugo čekanje s otvaranjem PR-a (branch je previše zastario)
- Dvije osobe mijenjaju isti zajednički element (uvodni tekst, naslovnicu)

Rješenje: **kraći ciklusi rada** — manji PR-ovi, češće merганje.

---

## 9.6 Nakon mergea — što dalje?

Nakon što je tvoj PR mergean:

1. Prebaci se na main i napravi pull (9.4)
2. Kreiraj novi branch za sljedeći zadatak (Poglavlje 6)
3. Počni pisati sljedeće poglavlje (Poglavlje 7)

Radni ciklus se ponavlja. Svaki PR je jedna zatvorena cjelina — jasna, pregledana i dokumentirana.

---

## 9.7 Git CLI alternativa za merge

**i Napomena:** Merge kroz CLI preporučujemo samo maintainerima s iskustvom. Za svakodnevni rad, GitHub sučelje iz poglavlja 9.3 je jednostavnije i sigurnije.

```
Prebaci se na main
git checkout main

Preuzmi najnovije promjene
git pull origin main

Spoji branch u main
git merge ana-kovac/uvod

Pošalji na GitHub
git push origin main

Obriši lokalni branch (opcionalno)
git branch -d ana-kovac/uvod
```

---

## ☑ Što smo naučili u ovom poglavlju

Koncept	Objašnjenje
<b>Merge</b>	Spajanje radnog brancha u main nakon odobrenog pregleda
<b>Merge commit</b>	Najjednostavniji tip mergea — preporučen za početnike
<b>Delete branch</b>	Brisanje brancha nakon mergea — čisti repozitorij
<b>Konflikt</b>	Nastaje kada isti redci iste datoteke postoje u dvije verzije
<b>Resolve conflicts</b>	GitHub alat za ručno rješavanje konflikata u pregledniku
<b>Pull nakon mergea</b>	Lokalni main se mora ručno ažurirati nakon online mergea

💡 **Zlatno pravilo:** Merge je završetak jednog ciklusa rada, ne kraj projekta. Nakon svakog mergea odmah kreiraš novi branch za sljedeći zadatak. Repozitorij je živ — uvijek postoji aktivan posao u tijeku.

---

<- Prethodno poglavlje: [Poglavlje 8 – Pull Request](#) Sljedeće poglavlje: [Poglavlje 10 – Word DOCX template: koncept](#) ->

## Poglavlje 10 - Word DOCX template: koncept

🕒 **Procijenjeno vrijeme čitanja:** 10 minuta 📋 **Preduvjet:** Razumijevanje osnovnog toka rada (Poglavlja 1–9)

---

### 10.1 Problem koji template rješava

Zamisli da pet osoba piše pet poglavlja u Obsidianu. Svako poglavlje je Markdown tekst — bez fontova, bez boja, bez margina. Kada Quarto spoji sve u jedan Word dokument, tko odlučuje kako će izgledati finalni dokument?

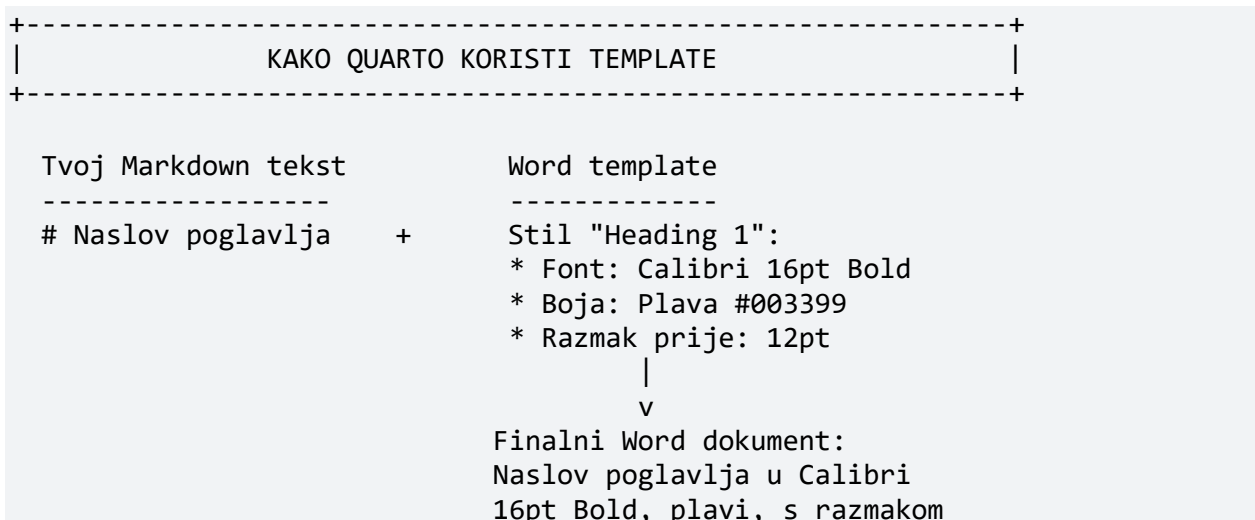
**Odgovor: Word template.**

Bez predloška, Quarto bi primijenio generički izgled koji ne odgovara standardima projekta. S predloškom, svaki put dobivaš identičan, profesionalno formatiran dokument — bez obzira tko je pisao koje poglavlje i s kojeg računala je pokrenut render.

---

## 10.2 Kako funkcionira reference-doc mehanizam

Quarto ne čita template kao “uzorak koji treba kopirati” — on čita **stilove** definirane u predlošku i primjenjuje ih na generirani dokument.



Quarto čita oznake iz Markdowna (#, ##, \*\*, tablice...) i za svaku oznaku pronalazi odgovarajući stil u predlošku. **Sadržaj dolazi iz Markdowna, oblik dolazi iz predloška.**

---

## 10.3 Gdje se template nalazi

U repozitoriju, template se nalazi na lokaciji:

```
EU-Project-Template/
└─ templates/
 └─ EU-dokument-template.docx <- Word predložak
```

Quarto zna gdje je template jer je putanja definirana u `_quarto.yml` konfiguracijskoj datoteci:

```
format:
 docx:
 reference-doc: templates/EU-dokument-template.docx
```

Ova veza je već postavljena — **ne trebaš ništa mijenjati** da bi template bio primijenjen. Svaki put kada pokreneš render, Quarto automatski koristi ovaj predložak.

---

## 10.4 Koje stilove template definira

Word predložak definira izgled svakog elementa dokumenta:



Markdown element	Word stil u predlošku
# Naslov	Heading 1
## Podnaslov	Heading 2
### Sekcija	Heading 3
Normalni tekst	Normal / Body Text
<b>**podebljano**</b>	Bold (unutar stila)
Tablice	Table Grid ili prilagođeni stil
Slike	Figure
Numeracija stranica	Footer stil
Zaglavlje dokumenta	Header stil

💡 **Važna implikacija:** Ako u Obsidianu pišeš # Poglavlje 1 i u predlošku stil Heading 1 ima definiranu automatsku numeraciju, finalni dokument će automatski ispisati 1. Poglavlje 1 — bez da si ti ručno upisivao/la broj. Formatiranje i numeracija su potpuno automatski.

---

## 10.5 Kako modificirati template

⚠️ **Upozorenje:** Modificiranje predloška mijenja izgled **svakog budućeg dokumenta** generiranog iz ovog repozitorija. Ovu promjenu radi isključivo **maintainer projekta**, po dogovoru s timom ili naručiteljem.

Ako ipak trebaš promijeniti izgled:

### Korak 1: Otvori template u Wordu

Dvostruki klik na templates/EU-dokument-template.docx — otvara se u Microsoft Wordu.

### Korak 2: Uredi stilove, ne sadržaj

Template ne sadrži pravi sadržaj dokumenta — sadrži samo ogledne primjere teksta koji pokazuju kako svaki stil izgleda.

**Što mijenjati:** - Otvori **Home -> Styles panel** (ili Alt+Ctrl+Shift+S) - Desni klik na stil koji želiš promijeniti (npr. Heading 1) - Odaberi **“Modify...”** - Promijeni font, veličinu, boju, razmak...

**Što ne mijenjati:** - Naziv stila (npr. Heading 1 mora ostati Heading 1 — Quarto ga traži po imenu) - Strukturu dokumenta

### Korak 3: Spremi i commitaj

1. Spremi datoteku u Wordu (Ctrl+S)
2. U Obsidianu: Stage All -> Commit -> Push

### 3. Otvori PR kao i za svaku drugu promjenu

💡 **Savjet:** Kada modificiraš template, u commit poruci napiši točno što si promijenio/la: npr. Template: promjena fonta naslova na Arial 14pt. Ovo je važno jer svaka promjena predloška utječe na sve buduće renderove.

---

## 10.6 Što se dogodi ako template nedostaje

Ako Quarto ne pronađe template na definiranoj putanji, render će i dalje raditi — ali s generičkim Word stilovima. Finalni dokument neće izgledati prema standardima projekta.

GitHub Actions će u logu prikazati upozorenje:

```
Warning: reference-doc 'templates/EU-dokument-template.docx' not found.
Using default styles.
```

Ako vidiš ovo upozorenje — provjeri je li datoteka na ispravnoj lokaciji i je li commitana u repozitorij.

---

## 10.7 Veza između template i finalnog dokumenta — sažetak

-----+   Markdown   sadržaj     # Naslov   Tekst...   - stavka  -----+ +-----+   Pišeš	+	-----+   Word   template     Heading 1:   Calibri 16pt   Blue     Normal:   Times 11pt  -----+ +-----+   Ne diraš	=	-----+   Finalni Word   dokument     NASLOV   (Calibri 16pt Blue)     Tekst u Body Text   stilu...  -----+ +-----+   Automatski rezultat
----------------------------------------------------------------------------------------------------------------	---	----------------------------------------------------------------------------------------------------------------------------------------------------	---	------------------------------------------------------------------------------------------------------------------------------------------------------------------------

Kao autor, tvoj jedini posao je **ispravno koristiti Markdown oznake**. Sve ostalo — fontovi, boje, margine, numeracija stranica, zaglavlja i podnožja — dolazi automatski iz predloška.

---

### ☑ Što smo naučili u ovom poglavlju

Koncept

Objašnjenje

**Word template**

.docx datoteka koja definira stilove finalnog dokumenta

**reference-doc**

Quarto mehanizam kojim se template primjenjuje na output

Koncept	Objašnjenje
<b>Stilovi</b>	Definirani u predlošku — Heading 1, Normal, Table Grid itd.
<b>Sadržaj vs. oblik</b>	Markdown = sadržaj; template = oblik. Nikad se ne miješaju
<b>Tko mijenja template</b>	Isključivo maintainer, po dogovoru, s commit porukom
<b>Ključna poruka ovog poglavlja:</b> Kao autor, ne brineš se za formatiranje. Pišeš ispravne Markdown oznake, a predložak se brine za sve ostalo. Konzistentnost između poglavlja različitih autora je zajamčena.	

---

<- **Prethodno poglavlje:** [Poglavlje 9 – Merge u main](#) **Sljedeće poglavlje:** [Poglavlje 11 – GitHub Actions: pregled automatizacije](#) ->

## Poglavlje 11 - GitHub Actions: pregled automatizacije

 **Procijenjeno vrijeme čitanja:** 10 minuta  **Preduvjet:** Razumijevanje toka rada (Poglavlja 1–10)

### 11.1 Što su GitHub Actions?

**GitHub Actions** je sustav automatizacije ugrađen direktno u GitHub. Kada se nešto dogodi u repozitoriju — primjerice netko pusha promjene — GitHub Actions automatski pokreće unaprijed definirane zadatke.

#### Analogija

Zamisli automatsku praonicu rublja u praonici samousluge: - Ti staviš rublje i pritisneš gumb (= push na GitHub) - Stroj automatski pere, ispire i centrifugira (= GitHub Actions pokreće korake) - Na kraju izvadi čisto rublje (= generiran Word dokument)

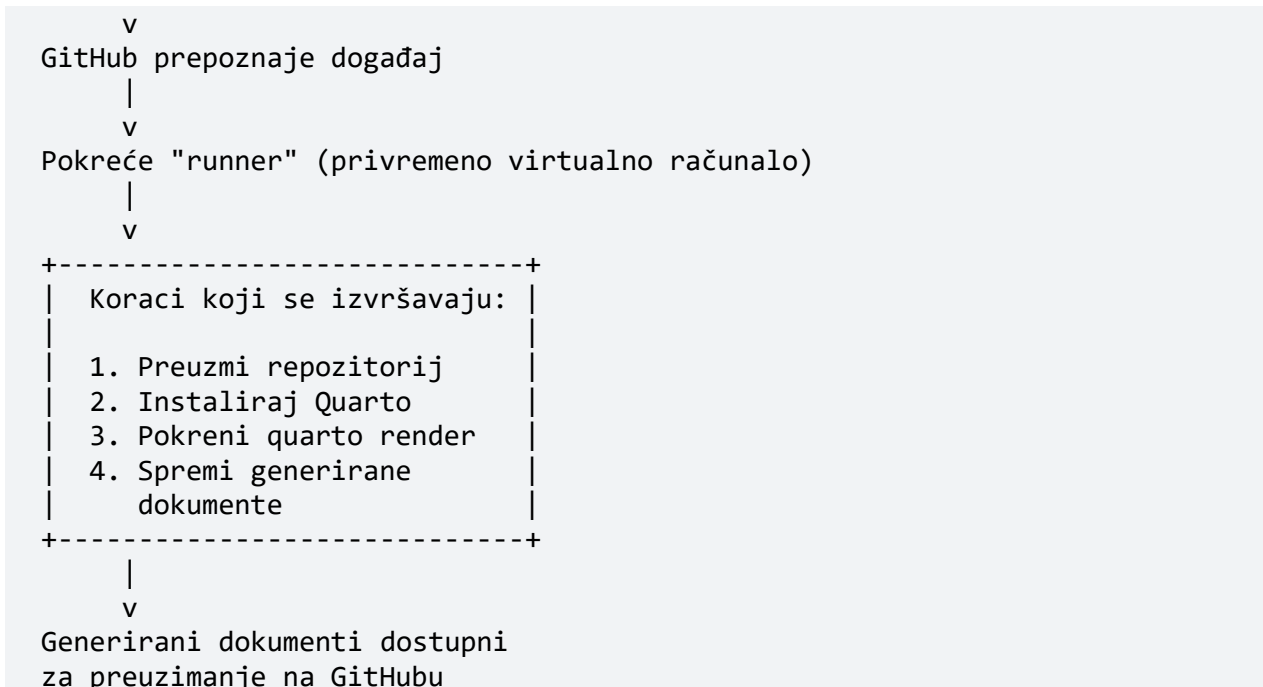
Ti ne moraš znati kako stroj radi iznutra — samo koristiš rezultat.

#### Što Actions radi u ovom projektu

Svaki put kada se promjene mergaju u main granu:

```
+-----+
| AUTOMATSKI TOK GITHUB ACTIONS |
+-----+

Push/Merge -> main
|
```



**Rezultat:** Svaki push na main automatski producira svježiju verziju finalnog dokumenta — bez da itko ručno pokreće Quarto na svom računalu.

---

## 11.2 Gdje se nalazi konfiguracija

Konfiguracija GitHub Actions nalazi se u posebnom folderu repozitorija:

```
EU-Project-Template/
└─ .github/
 └─ workflows/
 └─ render.yml <- datoteka koja definira automatizaciju
```

**Ne dirati:** Cijeli `.github/` folder je isključiva odgovornost maintainera. Greška u YAML konfiguraciji može onemogućiti cijelu automatizaciju. Ako misliš da nešto treba promijeniti — javi maintaineru.

### Kako izgleda workflow datoteka (samo za razumijevanje)

```
name: Render Quarto Documents

on:
 push:
 branches: [main] # <- pokreće se samo na main grani

jobs:
 render:
 runs-on: ubuntu-latest # <- GitHub koristi Linux virtualno računalo
 steps:
```

```
- uses: actions/checkout@v4 # preuzima repozitorij
- uses: quarto-dev/quarto-actions/setup@v2 # instalira Quarto
- run: quarto render # pokreće render
- uses: actions/upload-artifact@v4 # sprema generirane dokumente
```

Ne trebaš razumjeti svaki redak — dovoljno je znati da ova datoteka postoji i što radi u cjelini.

---

## 11.3 Praćenje statusa Actions — gdje gledati

### Zelena kvačica / Crveni X na GitHubu

Nakon svakog pusha na main, pored commit poruke na GitHubu pojavljuje se ikona:

Ikona	Značenje
	Žuta točka      Action je pokrenut, u tijeku
	Zelena kvačica      Action završio uspješno, dokumenti su generirani
	Crveni X      Action nije uspio — nešto je krenulo naopako

### Kako otvoriti Actions log

1. Na GitHubu, klikni na karticu **“Actions”** u gornjem izborniku repozitorija
2. Vidiš popis svih pokretanja — svaki push na main je jedan unos
3. Klikni na posljednje pokretanje

+-----+   Actions / Render Quarto Documents   +-----+			
[V]	Render Quarto Documents	#42	3 min ago
[V]	Render Quarto Documents	#41	2 hours ago
Ne	Render Quarto Documents	#40	yesterday
+-----+			

4. Klikni na pokretanje da vidiš detalje
5. Klikni na **“render”** u lijevom izborniku da vidiš log koraka

---

## 11.4 Čitanje loga — što tražiti

Log izgleda kao dugačak ispis teksta. Ne moraš čitati sve — traži **crvene linije s greškama**.

### Uspješan log

```
[V] Set up job
[V] Checkout repository
[V] Setup Quarto
[V] Run quarto render
```

```
Rendering docs/01-uvod.md
Rendering docs/02-metodologija.md
Output created: output/EU-izvjesce.docx
[V] Upload artifacts
[V] Complete job
```

## Log s greškom

```
[V] Set up job
[V] Checkout repository
[V] Setup Quarto
Ne Run quarto render
 Rendering docs/01-uvod.md
 ERROR: YAMLException: bad indentation of a mapping entry
 at line 3, column 5
 Execution halted
Ne Upload artifacts (skipped)
Ne Complete job
```

## Kako čitati poruku greške

Poruka greške obično sadrži: - **Vrstu greške** (YAMLException, FileNotFoundException, ParseError...) - **Datoteku** gdje je greška (docs/01-uvod.md) - **Redak i stupac** (at line 3, column 5)

Ove informacije su dovoljne da lociraš i popraviš problem.

---

## 11.5 Najčešće greške i kako ih prepoznati

Poruka greške	Uzrok	Rješenje
YAMLException: bad indentation	Razmaci u YAML zaglavlju nisu ispravni	Provjeri zaglavlje .md datoteke — ne koristi Tab, samo razmake
FileNotFoundException: template not found	Predložak nije na ispravnoj putanji	Provjeri je li templates/EU-dokument-template.docx commitana
ParseError: unexpected character	Poseban znak u Markdownu koji Quarto ne može parsirati	Pronađi redak naveden u grešci, provjeri znakove
Error: Process completed with exit code 1	Generička greška — gledaj prethodne linije loga	Skrolaj gore u logu — greška je opisana nekoliko redaka iznad

💡 **Savjet:** Kopiraj poruku greške i pošalji je maintaineru — s imenom datoteke i brojem retka. To je sve što je potrebno za dijagnozu.

---

## 11.6 Preuzimanje generiranih dokumenata

Kada je Action uspješno završio, generirani dokumenti su dostupni kao **Artifacts** (privremene datoteke koje GitHub čuva određeni broj dana).

### Kako preuzeti dokument

1. Na GitHubu, klikni karticu **“Actions”**
2. Klikni na posljednje uspješno pokretanje (zelena kvačica)
3. Skrolaj do dna stranice — sekcija **“Artifacts”**

Artifacts			
[ ]	EU-izvjesce-output	45.2 MB	v

4. Klikni na naziv Artifacta za preuzimanje ZIP datoteke
5. Raspakiraj ZIP — unutra je finalni .docx dokument

**i Napomena:** GitHub čuva Artifacts određeni broj dana (obično 90). Ako projekt ima duži vijek, maintainer može podesiti automatsko objavljivanje dokumenata na GitHub Pages ili drugoj lokaciji — ali to je naprednija konfiguracija.

---

## 11.7 Što korisnik treba znati, a što ne

### Trebaš znati

- Gdje gledati status Actions (kartica “Actions”)
- Razliku između zelene kvačice i crvenog X-a
- Kako otvoriti log i pronaći poruku greške
- Kako preuzeti generirani dokument iz Artifacts

### Ne trebaš znati (to je posao maintainera)

- Kako pisati ili mijenjati YAML workflow datoteke
  - Kako konfigurirati GitHub runners
  - Kako postavljati environment varijable i secrets
  - Kako mijenjati trigger uvjete (kada se Action pokreće)
-

## ☑ Što smo naučili u ovom poglavlju

Koncept	Objašnjenje
<b>GitHub Actions</b>	Sustav automatizacije koji se pokreće na svaki push u main
<b>Workflow</b>	YAML datoteka koja definira što se automatski radi
<b>Runner</b>	Privremeno virtualno računalo koje izvršava korake
<b>Artifact</b>	Generirani dokument dostupan za preuzimanje na GitHubu
<b>Log</b>	Ispis svih koraka — tražimo crvene linije za dijagnozu
<b>Zelena/crvena ikona</b>	Brzi vizualni signal o uspjehu ili neuspjehu rendera

💡 **Ključna poruka:** GitHub Actions radi u pozadini, automatski, bez tvoje intervencije. Tvoj jedini zadatak je prepoznati kada nešto nije prošlo (crveni X), pročitati poruku greške i javiti maintaineru — ili popraviti svoju .md datoteku ako greška pokazuje na nju.

---

<- **Prethodno poglavlje:** [Poglavlje 10 – Word DOCX template](#) **Sljedeće poglavlje:** [Poglavlje 12 – Quarto render: lokalni pregled dokumenta](#) ->

## Poglavlje 12 - Quarto render: lokalni pregled dokumenta

🕒 **Procijenjeno vrijeme:** 15–20 minuta 📋 **Preduvjet:** Quarto instaliran i provjeren (Poglavlje 2.3), repozitorij kloniran (Poglavlje 4)

---

### 12.1 Zašto renderirati lokalno?

GitHub Actions automatski renderira dokument svaki put kada se pusha na main — to smo vidjeli u Poglavlju 11. Zašto bi onda uopće pokretao/la Quarto lokalno na svom računalu?

#### Tri razloga:

1. **Brza provjera prije pusha** — vidiš kako će finalni dokument izgledati *prije* nego pošalješ promjene kolegama. Bolje uhvatiti grešku lokalno nego čekati da Actions javi problem na GitHubu.
2. **Rad bez interneta** — lokalni render ne zahtijeva GitHub. Možeš raditi i pregledavati rezultate i bez veze s mrežom.
3. **Brže iteracije** — lokalni render je trenutno. GitHub Actions treba 2–5 minuta. Ako testiraš izgled tablice ili slike, puno je brže renderirati lokalno.



💡 **Pravilo palca:** Lokalni render je za **osobnu provjeru**. GitHub Actions je za **službeni output** koji ide timu.

---

## 12.2 Otvaranje terminala u folderu repozitorija

Sve Quarto naredbe pokrećemo iz **Command Prompta** unutar foldera repozitorija.

### Windows

1. Otvori **File Explorer**
2. Navigiraj do C:\Projekti\EU-Project-Template
3. U adresnoj traci klikni jednom, upiši cmd, pritisni **Enter**

Otvora se Command Prompt s ispravnom lokacijom:

```
C:\Projekti\EU-Project-Template>
```

💡 **Alternativa:** U Obsidianu možeš otvoriti terminal direktno — Ctrl+P -> upiši terminal -> “Open terminal here”. Ako plugin nije instaliran, koristi File Explorer metodu iznad.

### macOS

1. Otvori **Finder**
2. Navigiraj do foldera repozitorija
3. Desni klik na folder -> “**New Terminal at Folder**”

---

## 12.3 Osnovne Quarto naredbe

### Render jedne datoteke

Renderira jednu .md datoteku u sve formate definirane u \_quarto.yml:

```
quarto render docs/01-uvod.md
```

### Render jedne datoteke u specifičan format

Renderira samo u Word dokument, bez obzira na ostale formate u konfiguraciji:

```
quarto render docs/01-uvod.md --to docx
```

Ostali dostupni formati:

```
quarto render docs/01-uvod.md --to html
quarto render docs/01-uvod.md --to pdf
```

⚠️ **PDF napomena:** Render u PDF zahtijeva instaliran LaTeX (tinytex). Ako ti PDF nije potreban, koristi docx ili html. Instalacija tinytex-a: `quarto install tinytex`

## Render cijelog projekta

Renderira sve datoteke definirane u `_quarto.yml` odjednom:

```
quarto render
```

💡 **Savjet:** Za svakodnevnu provjeru vlastitog poglavlja koristi render jedne datoteke — brže je. Render cijelog projekta koristi samo kada trebaš vidjeti kako sve zajedno izgleda.

## Preview — pregled u pregledniku

Preview pokreće lokalni web server i otvara dokument u pregledniku. Automatski se ažurira svaki put kada spremiš datoteku:

```
quarto preview docs/01-uvod.md
```

```
Preparing to preview
[✓] docs/01-uvod.md
Watching files for changes
Browse at http://localhost:4848/
```

Otvori preglednik na adresi `http://localhost:4848/` — vidiš live pregled dokumenta.

Za zaustavljanje pregleda pritisni `Ctrl+C` u Command Promptu.

💡 **Kada koristiti preview vs. render? - Preview** — dok pišeš i hoćeš vidjeti promjene odmah (HTML prikaz) - **Render** — kada hoćeš finalnu `.docx` datoteku za provjeru izgleda u Wordu

---

## 12.4 Gdje se nalaze generirani dokumenti

Nakon lokalnog rendera, Quarto sprema output u folder definiran u `_quarto.yml`. Najčešće:

```
EU-Project-Template/
├─ output/ <- generirani dokumenti idu ovdje
│ ├── 01-uvod.docx
│ ├── 02-metodologija.docx
│ └─ EU-projekt-kompletno.docx
```

Otvori `.docx` datoteku dvostrukim klikom — otvara se u Microsoft Wordu s primijenjenim stilovima iz predloška.

⚠️ **Važno:** Folder `output/` je naveden u `.gitignore` — generirani dokumenti se **ne commitaju** u repozitorij. Svaki put se generiraju iznova iz izvora. Ovo je namjerno: jedini “izvor istine” su `.md` datoteke, ne generirani outputi.

---

## 12.5 Razumijevanje \_quarto.yml — osnove

Datoteka \_quarto.yml je “mozak” Quarto projekta — definira što se renderira, kako i kamo. Nalazi se u korijenu repozitorija.

**Ne mijenjaj ovu datoteku bez dogovora s maintainerom.**

Evo što znače ključni dijelovi (samo za razumijevanje):

```
project:
 type: book # <- tip projekta: book = više datoteka u jedan do
kument

book:
 title: "EU Projektno izvješće"
 author: "Projektni tim"
 chapters:
 - docs/01-uvod.md # <- redoslijed poglavlja u finalnom dokumentu
 - docs/02-metodologija.md
 - docs/03-rezultati.md

format:
 docx:
 reference-doc: templates/EU-dokument-template.docx # <- Word predložak
 html:
 theme: default # <- tema za HTML output
```

### Što iz ovoga trebaš znati

Postavka	Što znači za tebe
chapters	Redoslijed u kojem se poglavlja pojavljuju u finalnom dokumentu — ako fali tvoja datoteka, poglavlje neće biti u outputu
reference-doc	Putanja do Word predloška — mora biti ispravna
format	Koji formati se generiraju — docx, html, pdf

💡 **Česta situacija:** Napisao/la si novu datoteku, renderiraš — i ne vidiš je u finalnom dokumentu. Razlog: datoteka nije dodana u chapters listu u \_quarto.yml. Javi maintaineru da je doda.

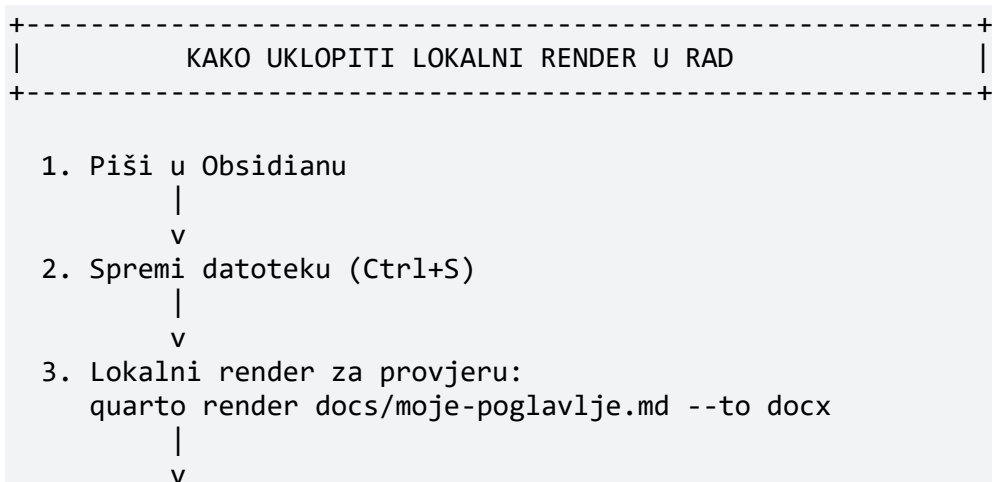
---

## 12.6 Razlika između lokalnog rendera i GitHub Actions

	Lokalni render	GitHub Actions
<b>Kada se</b>	Ručno, po potrebi	Automatski, na svaki push u main

	Lokalni render	GitHub Actions
<b>po kre će</b>		
<b>Tk o ga po kre će</b>	Ti, na svom računalu	GitHub, na virtualnom računalu
<b>Re zul tat</b>	Lokalna datoteka u output/	Artifact dostupan za preuzimanje
<b>Brz ina</b>	Trenutno (sekunde)	2–5 minuta
<b>Na mj en a</b>	Osobna provjera	Službeni output za tim
<b>Co m mit anj e out put a</b>	Ne commitaj	GitHub to radi automatski

## 12.7 Tipičan tijek rada s lokalnim renderom



4. Otvori output/moje-poglavlje.docx u Wordu  
-> Izgleda dobro? Nastavi.  
-> Nešto nije u redu? Vрати se u Obsidian i ispravi.  
|  
v
5. Commit + Push (Poglavlje 7)  
|  
v
6. GitHub Actions automatski renderira  
službenu verziju (Poglavlje 11)

---

## 12.8 Najčešće greške pri lokalnom renderu

Greška	Uzrok	Rješenje
quarto: command not found	Quarto nije instaliran ili PATH nije ažuriran	Restart računala, provjeri instalaciju (Pogl. 2.3)
ERROR: YAMLException	Greška u YAML zaglavlju .md datoteke	Provjeri zaglavlje — nema Tab-ova, ima zatvorenu ---
File not found: templates/...	Template nije na očekivanoj putanji	Provjeri je li templates/ folder prisutan i nije li u .gitignore
Pandoc not found	Quarto koristi Pandoc koji nije instaliran	Reinstaliraj Quarto — Pandoc dolazi u paketu
Prazan output folder	Render je uspio ali nisi u pravom folderu	Provjeri jesi li pokrenuo/la naredbu iz korijena repozitorija

---

### ☒ Što smo naučili u ovom poglavlju

Koncept	Objašnjenje
<b>Lokalni render</b>	Ručno pokretanje Quarta na svom računalu za osobnu provjeru
<b>quarto render</b>	Renderira jednu datoteku ili cijeli projekt
<b>quarto preview</b>	Live pregled u pregledniku, ažurira se automatski
<b>--to docx</b>	Zastavica za odabir formata outputa
<b>_quarto.yml</b>	Konfiguracijska datoteka — ne mijenjaj bez maintainera
<b>output/ folder</b>	Gdje završavaju lokalno generirani dokumenti — ne commitati

💡 **Ključna poruka:** Lokalni render je tvoj osobni alat za provjeru — koristi ga slobodno i često. Ne commitaj generirane dokumente. Službeni output uvijek dolazi iz GitHub Actions, automatski, nakon mergea u main.

---

<- **Prethodno poglavlje:** [Poglavlje 11 – GitHub Actions](#) **Sljedeće:** [Dodaci – Cheat sheet, Rječnik pojmova, Top 10 grešaka](#) ->

## Poglavlje 13 - Dodaci: Cheat Sheet, Rjecnik i Rjesavanje gresaka

Ovo poglavlje služi kao tvoja “prva pomoć”. Ne moraš ga čitati od početka do kraja. Spremi ga (ili isprintaj) kako bi mu se mogao/la brzo vratiti kada zaboraviš neku naredbu, kada čuješ nepoznat izraz ili kada sustav javi grešku.

### 13.1 Brzi referentni karton (Cheat Sheet)

Sve najvažnije akcije koje radiš svaki dan, sažete na jednom mjestu.

#### *Cheat Sheet*

Akcija	Obsidian Git (Paleta: Ctrl+P)	Terminal (Git CLI)
<b>Preuzmi najnovije stanje</b>	Git: Pull	git pull origin main
<b>Napravi svoju radnu kopiju</b>	Git: Create branch	git checkout -b ime/zadatak
<b>Provjeri gdje se nalaziš</b>	Pogledaj u donji desni kut Obsidiana	git branch
<b>Zabilježi promjenu</b>	Panel desno -> + -> Upiši poruku -> Commit	git add . git commit -m “Poruka”
<b>Pošalji na GitHub</b>	Git: Push	git push origin ime-grane
<b>Živi pregled dokumenta</b>	(Nema u Obsidianu)	quarto preview
<b>Generiraj Word na računalu</b>	(Nema u Obsidianu)	quarto render

### 13.2 Rječnik pojmova (Što znači ta riječ?)

IT industrija prepuna je engleskih izraza. Evo jednostavnih prijevoda onoga što znače u našem svakodnevnom radu:

- **Branch (Grana):** Tvoja izolirana, sigurna radna kopija projekta. Mjesto gdje slobodno pišeš i griješiš bez utjecaja na službeni dokument.
- **Commit:** Čin trajnog spremanja (bilježenja) jedne logične cjeline teksta, uvijek popraćen kratkom porukom koja objašnjava što si napravio/la.

- **Docs-as-Code:** Pristup u kojem dokumentaciju pišemo istim alatima kojima programeri pišu računalne programe (tekst, verzije, automatizacija).
- **Git:** Softver koji radi u pozadini na tvom računalu i pamti cijelu povijest svake izmjene u tvojim datotekama (tvoj “vremenski stroj”).
- **GitHub:** Web stranica na kojoj se čuva glavna, zajednička kopija tvog projekta i gdje odobravaš tuđi rad.
- **Main (Glavna grana):** Službena, finalna verzija tvog dokumenta. Nikada ne pišemo direktno u nju.
- **Merge (Spajanje):** Čin prepisivanja tvog odobrenog rada s tvoje radne grane u službenu main granu.
- **Pull Request / PR:** Službena molba kolegama koja kaže: “Završio sam, molim vas pregledajte moj rad i spojite ga u glavnu verziju.”
- **Pull (Povlačenje):** Preuzimanje najnovijih izmjena s GitHuba na tvoje računalo. (Radi to svako jutro!)
- **Push (Guranje/Slanje):** Slanje tvojih zabilježenih (commit) promjena s računala na GitHub.
- **Render:** Proces u kojem Quarto uzima tvoj običan tekst, provlači ga kroz Word predložak i “isprinta” gotov dokument.
- **Vault (Trezor):** Obsidianov naziv za glavni folder na tvom računalu u kojem se nalazi cijeli tvoj projekt.
- **YAML (Metapodaci):** Blok teksta na samom vrhu index.md datoteke (ili u \_quarto.yml) koji sadrži podatke o dokumentu (naslov, autor, datum).

## 13.3 Top 10 najčešćih grešaka (Troubleshooting)

Svi griješe, posebno na početku. Ovdje je popis 10 najčešćih problema i kako ih riješiti.

### 1. “Nema mog novog poglavlja u Wordu!”

- **Simptom:** Napisao/la si predivan tekst u datoteku 04-zakljucak.md, ali Quarto ga nije ubacio u gotov Word.

- **Rješenje:** Zaboravio/la si reći Quarto da to poglavlje postoji. Otvori datoteku \_quarto.yml i dodajte 04-zaključak.md na popis pod sekcijom chapters.

## 2. Slike se ne prikazuju (crveni križić u Quarto logu)

- **Simptom:** Quarto javlja grešku File not found.
- **Rješenje:** Provjeri jesi li zaista ubacio/la sliku u mapu Slike/ u Obsidianu. Zatim provjeri velika i mala slova. Git razlikuje velika i mala slova! Slike/Graf.png nije isto što i Slike/graf.png.

## 3. “GitHub ne prihvaća moju lozinku” (Authentication failed)

- **Simptom:** Kada pokušaš napraviti Push ili Clone, terminal uporno traži lozinku i javlja grešku iako si upisao/la točnu lozinku tvog GitHub računa.
- **Rješenje:** GitHub više ne dopušta obične lozinke. Moraš koristiti onaj dugi kod (PAT - Personal Access Token) koji smo kreirali u Poglavlju 2.

## 4. “Ovaj dokument ima konflikte!” (Merge Conflict)

- **Simptom:** Prilikom pokušaja spajanja (Merge) na GitHubu gumb je siv i sustav prijavljuje konflikte.
- **Rješenje:** Ti i kolega mijenjali ste isti redak teksta istovremeno. Klikni “Resolve conflicts” na GitHubu, obriši linije s <<<<<< i ostavi samo onaj tekst koji treba preživjeti.

## 5. Gubitak Word formatiranja

- **Simptom:** Ručno si popravio/la marginu i font u gotovom Word dokumentu, ali sljedeći dan je sve opet po starom.
- **Rješenje:** Sjeti se zlatnog pravila iz Poglavlja 10: Gotov Word se **nikada** ne uređuje ručno. Sve vizualne promjene moraš napraviti u praznoj template.docx datoteci.

## 6. Kolege ne vide moje promjene

- **Simptom:** Ti vidiš tekst u Obsidianu, ali tvoj voditelj na GitHubu ne vidi ništa novo.
- **Rješenje:** Vjerojatno si zaboravio/la odraditi **Push**. Ako si samo kliknuo/la “Commit” (spremi), promjene su sigurne, ali su i dalje samo na tvom računalu. Moraš napraviti Push.

## 7. “Ne mogu napraviti Push na main!”

- **Simptom:** Pokušavaš poslati promjene, ali Git javlja grešku “Protected branch” ili “Permission denied”.



- **Rješenje:** Slučajno si počeo/la pisati tekst na glavnoj (main) grani, a ona je zaštićena. Otvori paletu (Ctrl+P), upiši Git: Create branch, napravi novu granu i Git će automatski prebaciti tvoj novi tekst na nju. Zatim napravi Push.

## 8. Quarto javlja “Invalid YAML” grešku

- **Simptom:** Crveni križić na GitHub Actions ili crveni tekst u terminalu koji spominje YAML.
- **Rješenje:** YAML je jako osjetljiv na razmake. Provjeri index.md i \_quarto.yml. Jesi li slučajno obrisao/la dvotočku (:)? Jesi li koristio/la “Tab” tipku umjesto razmaka? YAML koristi isključivo obične razmake!

## 9. Obsidian Git panel je nestao

- **Simptom:** Nema “Source Control” ikone na desnoj strani Obsidiana.
- **Rješenje:** Obsidian je vjerojatno iz sigurnosnih razloga blokirao plugine. Idi u Settings -> Community Plugins, isključi “Safe Mode” i provjeri je li “Obsidian Git” omogućen (Enabled).

## 10. Ne razumijem zašto se Quarto ruši

- **Simptom:** Crveni križić, čudna greška koja nema smisla, panika.
- **Rješenje:**
  1. Duboko udahni.
  2. Izgovori tri puta na glas: “GIT! GIT! GIT!”.
  3. Ne činite ništa
  4. Učinite nešto
  5. Kopiraj grešku iz loga i pošalji je svom administratoru.

## 13.4 Korištenje AI alata (Tvoj osobni Git asistent)

Ako se nađeš u problemu koji nije pokriven u “Top 10 grešaka”, tvoj najbolji prijatelj može biti umjetna inteligencija.

Alati poput **ChatGPT-a**, **Google Gemini-a** ili **Claude-a** iznimno su vješti i precizni u rješavanju problema s Gitom. Budući da je Git globalni industrijski standard, AI alati poznaju sve njegove naredbe do u detalj.

Ne moraš biti stručnjak da bi ih pitao/la za pomoć. Dovoljno je objasniti što želiš postići na običnom jeziku.

### **Primjer tvog pitanja (Prompt):**

*“Završio sam s pisanjem uvodnog poglavlja u svom editoru. Sada želim sve to spremiti. Kako ću točno napraviti commit i poslati to na GitHub preko terminala?”*

### **Primjer AI odgovora koji ćeš dobiti:**

1. Pripremi sve izmijenjene datoteke za spremanje:  
git add .
2. Zabilježi promjene (commit) uz kratku opisnu poruku:  
git commit -m “Završeno uvodno poglavlje”
3. Pošalji promjene na tvoju radnu granu na GitHubu (zamijeni ime-vase-grane stvarnim imenom):  
git push origin ime-vase-grane

**Savjet za greške:** Ako ti terminal izbací crveni tekst s greškom koju ne razumiješ, samo je **označi, kopiraj i zalijepi u AI alat** uz pitanje: *“Dobio sam ovu grešku u Gítu, kako da je riješim korak po korak?”*. AI će ti gotovo uvijek dati precizno rješenje unutar par sekundi.

**Kraj priručnika!** Želimo ti ugodan i produktivan rad u novom okruženju!